



EVERYONE READS THIS FIRST

Campus Placement and Training Cell

Advanced Java project, B.Sc. IT Semester V. Ten minutes here and you can explain the whole app.

2

portals: student,
officer

25

students

10

companies

14

drives

85

applications

146

automated tests

1. What the app is

SAY THIS

A college placement cell has to tell hundreds of students which company drives they can sit for, collect their applications, and move each one through shortlist, interview and offer. This app does that in one place. A student sees only the drives their own record qualifies them for, and when they do not qualify it says exactly why. The placement officer announces drives, tracks every candidate and reads live reports straight from the database.

It is a Jakarta EE 10 web application: Servlets and JSP on the front, EJB beans for the business rules, JPA with Hibernate and hand-written JDBC underneath, and MySQL 8 as the database.

2. Who exactly uses it

USER	HOW THEY GET AN ACCOUNT	WHAT THEY CAN DO	THEIR PAGES
Student	Registers themselves at <code>/register</code> , or the officer imports them in bulk.	Keep a profile and resume, browse drives, see eligibility with reasons, apply, withdraw, follow status and notes from the cell.	<code>/app/ ...</code>
Placement officer (admin)	Created by the college, never through the public form. Three exist in the demo data.	Add companies, announce and close drives, move candidates through the pipeline, shortlist in batches, import students, read reports, browse the database, read the audit trail.	<code>/admin/ ...</code>

What about the companies?

Recruiters do not log in. A company is a record the officer maintains, and the officer posts drives on its behalf, which is how a placement cell really works: the cell is the go-between. If the teacher asks "who puts up the job opportunities", the answer is the placement officer, from the Drives page.

3. How data gets in, and how it gets out

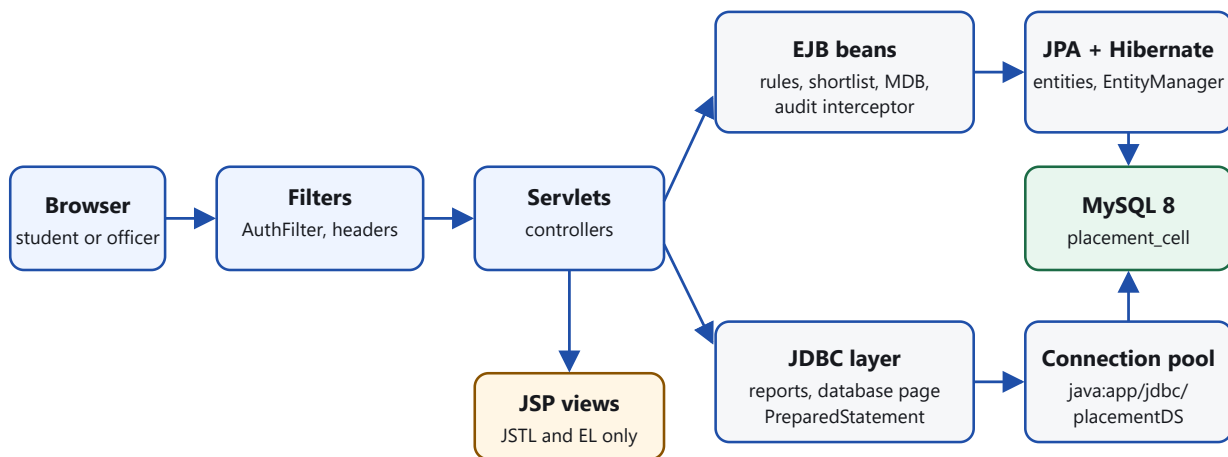
Data in

WHO	ACTION	LANDS IN TABLE
Student	Register	<code>app_user</code> , <code>student_profile</code>
Student	Edit profile, upload resume	<code>student_profile</code> and a file on disk
Student	Apply or withdraw	<code>job_application</code>
Officer	Add a company	<code>company</code>
Officer	Announce or close a drive	<code>drive</code>
Officer	Bulk import students	<code>app_user</code> , <code>student_profile</code>
Officer	Move a candidate, shortlist	<code>job_application</code> status and note
System	Every business write	<code>audit_log</code>
System	First start on an empty database	all tables, from <code>DataSeeder</code>

Data out

WHO	SCREEN	WHAT THEY SEE
Student	Drives board	Open drives, eligible or not, with the reason
Student	My applications	Status and the cell's latest note
Student	Resume link	Their own file, downloaded by the server
Officer	Overview	Totals, placed percentage, recent applications
Officer	Pipeline	Every application, filtered by drive and status, with a control to move it along
Officer	Reports	Branch and company numbers from plain JDBC, with the SQL shown
Officer	Database	Schema, rows and a read only SQL console
Officer	Audit	Who did what, how long it took
Both	Notification	A JMS message is queued on apply and on status change

4. Architecture in one picture



A servlet never writes HTML itself: it forwards to a JSP under WEB-INF, so no view can be opened by URL. Both data paths share one pool.

Read it left to right. A request meets the filters, then a servlet. The servlet asks an EJB (business rules and normal reads and writes through JPA) or the JDBC layer (reports and the database page). Both end at the same MySQL through the same server managed pool. The result goes back to a JSP, which produces the page.

5. Features at a glance

Student portal

- Self registration with a real email domain check
- Sign in with lockout after repeated failures
- Profile: branch, CGPA, backlogs, phone, resume upload
- Drives board that hides what you cannot apply for and says why
- Apply, withdraw, and follow status with the cell's notes
- A welcome panel after sign up and a getting started checklist on the overview
- Light and dark theme, remembered in a cookie

Officer portal

- Companies and drives with eligibility rules per drive
- Application pipeline and a batch shortlist
- Bulk import of students
- Reports read with plain JDBC, SQL shown beside the result
- Database page: schema, rows, guarded SQL console
- Audit trail, system page, and the Classroom JDBC page

6. Which course topic lives where

TOPIC FROM THE COURSE	WHERE IT IS IN THIS APP
Servlets, lifecycle, annotations	Every class in <code>web/</code> , declared with <code>@WebServlet</code>

RequestDispatcher	<code>Web.render</code> forwards every page to its JSP
Cookies	Remember-my-email, theme, last visit
HttpSession and listener	Signed-in user in the session, <code>SessionTracker</code> counts sessions
File upload and download	Resume upload, <code>ResumeDownloadServlet</code>
Non blocking I/O	<code>BulkImportServlet</code> reads its body with a <code>ReadListener</code>
JSP, actions, EL, JSTL	All views, scriptlets switched off in <code>web.xml</code>
EJB: stateless, stateful, singleton, MDB, interceptor	Services, <code>ShortlistBean</code> and <code>MockScoreBean</code> , <code>DataSeeder</code> , <code>NotificationMDB</code> , <code>AuditInterceptor</code>
JNDI	The datasource name and the stateful bean lookups
JPA with Hibernate	<code>entity/</code> and <code>persistence.xml</code>
JDBC	<code>jdbc/</code> , plus the classroom style <code>MockScoreBean</code>

7. Who explains what

PERSON	AREA	READ
Karan	The app itself: users, data in and out, demo, seed data, the student and officer flows	<code>01-karan-app-overview</code>
Yuvraj	MySQL connectivity and JDBC, the connectivity demo	<code>02-yuvraj-mysql-jdbc</code>
Tiya	Account creation, login, passwords, sessions, security	<code>03-tiya-auth-security</code>
Kinjal	JSP views, validation, resume upload, bulk import	<code>04-kinjal-views-validation</code>
Chandra	Entities, JPA, Hibernate, every EJB, tests	<code>05-chandra-entities-ejb</code>

Everyone also reads `06-viva-questions` (every question we expect, with an owner and an answer) and `07-demo-run-sheet` (the order of the presentation, with timings).

8. Logins for the demo

ROLE	EMAIL	PASSWORD	USE IT FOR
Officer	<code>tpo@campus.edu</code>	<code>Campus@2026</code>	The officer walkthrough
Student	<code>aarti.deshpande@campus.edu</code>	<code>Campus@2026</code>	Main student demo: eight applications, no offer yet, drives left to apply for
Student	<code>rahul.menon@campus.edu</code>	<code>Campus@2026</code>	A placed student (has an offer)
Student	<code>sneha.pillai@campus.edu</code>	<code>Campus@2026</code>	CGPA 9.1, a second live account

Every seeded account shares one password. That is for the demo only, and the hashes in the database are all different because each has its own salt.

9. Get it running on your laptop

1. Clone the repository and open the folder `placement-cell` in VS Code.
2. Start the MySQL 8 service. In MySQL Workbench, create the database and user once (the four lines are in `README.md`).
3. In PowerShell, once per machine: `.\setup.ps1` (downloads the server and JDK into `server\`).
4. Every time: `.\run.ps1`. On the morning of the presentation use `.\run.ps1 -Reseed` so every drive has a fresh deadline.
5. Open `http://localhost:8080/placement/`. Stop with `.\stop.ps1`.
6. Shortcut: double click `start.bat` in the folder. It builds, starts the server, deploys and opens the browser.

CAREFUL

The seeded dates are counted from the day the data is created. If the database was created days ago, drives can show "the last date to apply has passed". Run `.\run.ps1 -Reseed` to fix that.



KARAN

The app itself: users, data, flows and the demo

You open the presentation and explain what the app is, who uses it, how data moves, and you run the walkthrough.

1. Your role in one sentence

You explain the application as a whole, and you own the code that turns each screen into an action: the student and officer flows, the dashboards, and the demo data.

2. What you own

FILE	LINES	WHAT IT IS
<code>ejb/DataSeeder.java</code>	336	Singleton bean that fills an empty database: 3 officers, 25 students, 10 companies, 14 drives, 85 applications
<code>web/StudentDashboardServlet.java</code>	88	The student's landing page
<code>web/DriveBrowseServlet.java</code>	90	The drives board and a single drive, with the eligibility verdict for each
<code>web/ApplicationServlet.java</code>	127	Apply, withdraw, and the "my applications" list
<code>web/AdminDashboardServlet.java</code>	66	The officer's landing page and its totals
<code>web/AdminDriveServlet.java</code>	220	Announce, edit, open, close and delete drives, with validation
<code>web/AdminCompanyServlet.java</code>	126	The recruiter register
<code>web/AdminStudentServlet.java</code>	108	Student register, a student's record, suspend and unlock
<code>web/AdminApplicationServlet.java</code>	107	The pipeline: filter, and move a candidate's status with a note
<code>web/ShortlistServlet.java</code>	159	Batch shortlist using the stateful bean
<code>web/AuditServlet.java</code> , <code>web/SystemServlet.java</code>	100	The audit trail and the system page
<code>run.ps1</code> , <code>README.md</code> , <code>docs/team/</code>		How the app is started, reseeded and explained

3. The two minute explanation

SAY THIS

What it is. Every placement season a college has to match students to company drives. This app is that matching desk. It has two kinds of user: students and the placement officer.

Student side. A student registers with a real email and their academic record: branch, CGPA, backlogs. From then on the app compares that record with the rules on every drive. Drives they qualify for show an Apply button. Drives they do not qualify for show the exact reason, for example "minimum CGPA is 7.5, yours is 6.4". They apply, and then follow the status and the officer's notes.

Officer side. The officer maintains the companies, announces drives with their rules, moves candidates from applied to shortlisted to interview to selected, and can shortlist many at once. There are reports that come straight from the database with plain JDBC, a page that shows the database itself, and an audit trail of everything that changed.

Behind it. Servlets receive requests, EJB beans hold the business rules, JPA and Hibernate save the objects to MySQL, and JSP pages draw the screens. The reports use JDBC directly on the same connection pool. Each of us will now explain our own layer.

4. The story, step by step

A student's journey

- 1 **Register** at `/register`. Name, college email, password, roll number, branch, year, CGPA, backlogs, phone. The row goes to `app_user` and `student_profile`. The sign in page then shows a "Welcome, your account is ready" panel with the email filled in. (Tiya explains how.)
- 2 **Sign in** at `/login`. The servlet checks the password hash and stores a small `SessionUser` in the HttpSession.
- 3 **Dashboard** at `/app/dashboard`: profile summary, counts of your applications by status, how many open drives you can still apply to, and, for a new student, a getting started checklist (create account, finish profile, apply to a first drive) with a progress bar and the next button to press.
- 4 **Drives board** at `/app/drives`. `DriveBrowseServlet` asks `DriveService.check` for each open drive and shows the verdict.
- 5 **Apply**: open a drive, press Apply. `ApplicationServlet.doPost` checks the rules again, then `ApplicationService.apply` writes the row and queues a notification message.
- 6 **Follow it** at `/app/applications`: status and the cell's latest note. A student may withdraw.

The officer's journey

- 1 **Sign in** with an officer account. There is no public way to become an officer.

2 Overview at `/admin/dashboard` : how many students, how many placed, open drives, applications.

3 Companies and drives: add a recruiter, announce a drive with role, package, location, minimum CGPA, backlog limit, branches and dates. Every field is validated on the server.

4 Pipeline at `/admin/applications` : filter by drive and status, move a candidate on, write a note the student will see.

5 Shortlist at `/admin/shortlist` : tick many candidates, then commit them together. A stateful bean holds the selection between clicks.

6 Reports, Database, Audit: Yuvraj and Chandra explain these.

5. The demo data, and why it looks complete

`DataSeeder` is a singleton bean marked `@Startup` . On deploy it counts the accounts. If there are none, it fills the database. If there are some, it does nothing, so restarting never wipes real work.

TABLE	ROWS	WHAT IS IN IT
<code>app_user</code>	28	3 officers and 25 students. Every password is stored as its own salted PBKDF2 hash.
<code>student_profile</code>	25	Branches: 8 Computer Science, 6 Information Technology, 4 Electronics, 2 Statistics, 2 Mechanical, 2 Civil (one Civil student is suspended). CGPA runs from 5.6 to 9.3. Three have no phone number, so the profile warning appears.
<code>company</code>	10	Northwind Systems, Blue Harbour Analytics, Vertex Financial, Lumen Health, Kestrel Logistics, Orbit Semiconductors, Meridian Consulting, Paperkite Learning, Ironbridge Infrastructure, Saffron Retail Tech
<code>drive</code>	14	10 open with deadlines from 3 to 24 days away, and 4 already closed, so the archive has history. Rules differ: package 5.0 to 12.5 LPA, minimum CGPA 5.5 to 8.0, backlogs 0 to 3, different branch lists.
<code>job_application</code>	85	Every status appears: applied, shortlisted, interview, selected, rejected. 6 students hold an offer.

How the applications were made, in case you are asked

Six are written by hand to give the demo accounts a story. The rest come from simple arithmetic over the student and drive lists, and only for pairs where the student really meets the drive's rules, using the same three tests as the live app. Nothing is random, so the data is the same every time and the demo can be rehearsed. Every date is counted from today.

CAREFUL

Because dates are counted from the day the data was created, a database seeded last week has drives whose deadline has passed, and students cannot apply. Before the presentation run `.\run.ps1 -Reseed` . It empties the tables and lets the seeder fill them again with fresh dates.

6. Code walk in VS Code

- 1 **Open** `DriveBrowseServlet.java` . Point at the loop that calls `driveService.check(profile, drive)` for every drive and puts the map of verdicts on the request. Then `Web.render(request, response, "drives.jsp", "drives")` . Say: the servlet decides nothing, it asks the bean and hands the answer to a page.
- 2 **Open** `ApplicationServlet.java` , `doPost` . Show the order: find the drive, `check` the verdict, refuse if already applied or not eligible, then `applicationService.apply` , then a flash message and a redirect. Say: the button was hidden already, but the rules run again here because a form can be replayed or written by hand.
- 3 **Same file, `Web.redirect` after the write.** Say: after a POST we redirect instead of forwarding, so pressing refresh cannot apply twice (Post, Redirect, Get).
- 4 **Open** `AdminDriveServlet.java` . Show the block of `errors.put(...)` lines: role, package range, CGPA range, at least one branch, last date on or before the drive date.
- 5 **Open** `ShortlistServlet.java` . Show `basket()` : a JNDI lookup of `java:module/ShortlistBean` stored in the HttpSession. Say why it is not injected: a servlet is one shared object, a stateful bean must belong to one officer.
- 6 **Open** `DataSeeder.java` . Show `@Singleton @Startup` , the "count accounts" guard at the top of `seed()` , and the `meets` method used to make only valid applications.

7. Questions you will be asked

Q Who exactly uses this app?

Two kinds of people. Students, who register themselves, and placement officers, who are created by the college. Companies do not log in: the officer keeps their details and posts their drives, as a real placement cell does.

Q How does data get into the system?

Through forms. A student registration creates the account and profile. A student applying creates an application. An officer creating a company or a drive creates those rows. An officer can also import many students at once. Everything passes server side validation before it reaches the database, and the demo data is loaded once by the seeder.

Q How does data get out?

Students see a board of drives with their eligibility and their own applications. Officers see totals, a pipeline, reports read with JDBC, a browser for the database itself, and an audit trail. A student can also download their own resume.

Q Why can an officer not register on the public page?

Because the register form only ever creates a student. An officer account is a privilege, so it is created by the college. Only the seeder creates officers here. That keeps a student from choosing their own role.

Q Why are there no recruiter logins?

It was a scope decision. The cell is the go-between, so the officer posts drives for the company. A recruiter role would be a natural extension: a third role, its own pages, and an approval step by the officer.

Q A student says a drive is missing. Why?

The default board shows open drives. The board also has an "all" view. A drive whose last date has passed shows as not eligible with the reason "the last date to apply was ...", so nothing is hidden without explanation.

Q Is the eligibility check only on the page?

No. The page hides the button, and the servlet checks the rules again on the POST, because a page can be stale and a request can be hand written.

Q Where does the demo data come from, and is it real?

It is made up. The seeder creates it on the first start if the database is empty. It is deterministic, so it is the same every time, and the dates are counted from today so the drives are always current.

Q What did you not build?

Recruiter logins, email or SMS delivery of the notifications (a message is queued through JMS and handled by a bean, not sent by email), password reset by email, and confirmation links for new accounts. Two smaller gaps: nothing stops a student who already holds an offer from applying to more drives (the dashboard says the cell will be in touch about the policy), and the audit trail records the actor as "portal" instead of the individual officer, because sign in is done by the app's own filter rather than a server security realm. These are the honest next steps.

Q Which technologies are used and why?

Servlets and JSP for the web layer, EJB for business rules and transactions, JPA with Hibernate for ordinary reads and writes, JDBC for reports where SQL fits better than objects, MySQL as the database, GlassFish as the Jakarta EE server. Each one is a topic from the course.

8. If something goes wrong

SYMPTOM	FIX
Page will not load	Is MySQL running? Is the server up? Run <code>.\run.ps1</code> and wait for "deployed".

Drives all say "the last date has passed"	Stale seed dates. Run <code>.\run.ps1 -Reseed</code> .
"That form could not be verified"	The page sat open too long or the server restarted. Reload the page and try again.
Login fails for everyone	The database may be empty or half filled. <code>.\run.ps1 -Reseed</code> .



YUVRAJ

MySQL and JDBC: how the app connects to the database

You own database connectivity and reporting, and you run the connectivity demo. The teacher will test you hard here, so this brief is a script: what to open, what to click, what to say, and the queries that prove it.

1. Your role in one sentence

You explain and demonstrate how the application reaches MySQL: one JNDI datasource and connection pool, used by Hibernate and by hand-written JDBC, with the reports, the database browser and the classroom style JDBC page built on top.

2. What you own

Paths are under `src/main/java/com/campus/placement/` unless shown. Line counts are from `wc -l`.

FILE	LINES	WHAT IT IS
<code>jdbc/PlacementReportDao.java</code>	438	The JDBC layer: JNDI lookup, PreparedStatement, ResultSet, the manual transaction, DatabaseMetaData connection info
<code>jdbc/DatabaseExplorerDao.java</code>	366	Schema and row browsing through DatabaseMetaData, and the guarded read only SQL console
<code>jdbc/BranchReport</code> , <code>CompanyReport</code> , <code>CandidateRow</code>	84, 79, 95	Plain result carriers for the report rows (not JPA entities)
<code>jdbc/ConnectionInfo</code> , <code>ResultTable</code> , <code>TableDetail</code>	45, 98, 208	Carriers for connection facts, console results and table structure
<code>web/ReportsServlet.java</code>	116	Controller for <code>/admin/reports</code> : GET runs the reports, POST closes a drive in a transaction
<code>web/DatabaseExplorerServlet.java</code>	96	Controller for <code>/admin/database</code> : Structure, Rows and Query console tabs
<code>web/ClassroomJdbcServlet.java</code>	84	Controller for <code>/admin/classroom-jdbc</code> : validates input, then calls one of two bean methods
<code>ejb/MockScoreBean.java</code>	113	Stateful bean: <code>addScore</code> (classroom style) and <code>addScoreSafely</code> (pooled, PreparedStatement)
<code>src/main/webapp/WEB-INF/glassfish-resources.xml</code>	57	Defines the connection pool and the JNDI datasource. The only place the URL and password live
<code>views/admin-reports.jsp</code> , <code>admin-database.jsp</code> , <code>admin-classroom.jsp</code>	304, 374, 138	The three pages you demo (under <code>src/main/webapp/WEB-INF/</code>)
<code>src/test/.../jdbc/DatabaseConnectivityTest.java</code> , <code>QueryConsoleGuardTest.java</code>	182, 113	Tests against the real database, and tests of the console guard

Also point at, but do not own: `src/main/resources/META-INF/persistence.xml` (Hibernate reads the same datasource).

3. The 60 second explanation

SAY THIS

Our Java code never contains a database URL or a password. The URL and password sit in one server file, **glassfish-resources.xml**. GlassFish reads it and builds a **connection pool**: four MySQL connections opened up front, growing to at most 24. It publishes that pool under the JNDI name **java:app/jdbc/placementDS**.

The code just asks the naming service for that name and gets a **DataSource**. It calls **getConnection()** and is handed a pooled connection. It runs a **PreparedStatement** with question mark placeholders, reads the **ResultSet**, and **closes** everything, which returns the connection to the pool.

The same datasource is used twice. Hibernate gets it through **persistence.xml** for normal saving and loading. My JDBC code gets it directly for reports, the database browser and a manual transaction. One pool, one set of credentials, two ways of using it.

The one place a URL and password do appear in Java is **MockScoreBean.addScore**, on purpose: it is our copy of the classroom practical, kept so we can compare it with the proper way.

I will show you the chain in the code, then the same rows in the browser, then in MySQL itself.

4. Code walk in VS Code

Open these tabs in this order before you start. Line numbers are current as of this brief, so check them once after any edit.

```
JNDI name → connection pool → DataSource → Connection → PreparedStatement → ResultSet → close (back to the pool)
java:app/jdbc/placementDS → java:app/placementPool → ds.getConnection() → preparedStatement(sql) → executeQuery() → rs.next() → try with resources
```

- 1 **WEB-INF/glassfish-resources.xml , lines 38 to 55.** Point at the `jdbc-connection-pool` (line 38) with `steady-pool-size="4"` and `max-pool-size="24"` (lines 41 to 42), then the URL, user and password properties (lines 50 to 52), then the `jdbc-resource` on line 55 that publishes it as `java:app/jdbc/placementDS`.

Say: "This is the only file with the URL and password. It defines the pool and gives it a JNDI name. Change the password here and nothing in Java changes."

- 2 **META-INF/persistence.xml , lines 20 and 26.** Point at the Hibernate provider (line 20) and `<jta-data-source>java:app/jdbc/placementDS</jta-data-source>` (line 26).

Say: "Hibernate gets a name, not a URL. It is the same name as the pool above."

- 3 **PlacementReportDao.java , lines 66 to 93.** Point at `JNDI_NAME` (66), the `@Resource(lookup = JNDI_NAME)` field (73 to 74), and `lookupDataSource()` (91 to 93) with `new InitialContext().lookup(JNDI_NAME)`.

Say: "The annotation and the explicit lookup do the same thing. I kept the manual one so I can point at the actual JNDI call. The same name feeds Hibernate on the persistence.xml side."

- 4 **PlacementReportDao.branchReport() , lines 161 to 180.** Point at the try with resources header (163 to 165): `dataSource().getConnection()`, `connection.prepareStatement(BRANCH_SQL)`, `statement.executeQuery()`. Then the `while (rs.next())` loop (167) reading columns by name. Scroll up to `BRANCH_SQL` (138 to 154) and note the LEFT JOIN.

Say: "Five steps in order: datasource, connection, prepared statement, result set, close. Try with resources closes all three in reverse order even if the query throws. Closing the connection hands it back to the pool."

- 5 **PlacementReportDao.searchCandidates() , lines 252 to 299.** Point at `sql.append(" AND s.branch = ? ")` (272) and `parameters.add(branch)` (273), then the binding loop `statement.setObject(i + 1, parameters.get(i))` (297 to 299).

Say: "The SQL text never contains anything a user typed. Only question marks are added. The values are bound afterwards, so they can never be read as SQL."

- 6 **PlacementReportDao.closeDriveAndRejectPending() , lines 362 to 419.** Walk down: `setAutoCommit(false)` (378), first `executeUpdate()` (382), second (389), `commit()` (392), `rollback()` in the catch (398), and the `finally` (405 to 418) that restores auto commit and closes. Also show `@TransactionAttribute(NOT_SUPPORTED)` on line 56.

Say: "Closing a drive and rejecting its pending applications must happen together or not at all. Auto commit off, two updates, one commit. If the second update fails, rollback undoes the first. The finally block puts auto commit back before the connection returns to the pool, otherwise the next user would inherit a connection with auto commit off."

- 7 **PlacementReportDao.describeConnection() , lines 100 to 121, then DatabaseExplorerDao.describe() , lines 99 to 170.** First method: `connection.getMetaData()` and `meta.getURL()`, `getDriverName()`, `getUserName()`. Second: `getPrimaryKeys` (111), `getColumns` (117), `getImportedKeys` (133), `getIndexInfo` (147).

Say: "DatabaseMetaData lets the connection describe itself and the schema. Nothing on the Reports or Database pages is typed in. It is all asked of the live connection."

- 8 **DatabaseExplorerDao.runReadOnly() (216 to 256) and whyRefused() (261 to 281).** Point at the checks in order: no semicolon inside, must start with select, with, show, describe or explain (the pattern on line 62), must not contain a forbidden word (line 67), max 2000 characters. Then in the run: `connection.setReadOnly(true)` (235), `setQueryTimeout(TIMEOUT_SECONDS)` (238), `setMaxRows(MAX_ROWS)` (239).

Say: "Layer one is the keyword guard. Layer two is a read only connection. The rows are capped at 200 and a query is cancelled after 5 seconds. Password hash columns are replaced with (hidden) before they leave this class."

- 9 **MockScoreBean.java , addScore (63 to 78) beside addScoreSafely (81 to 91).** First, name the source: **Prac7-MarksEntry** is the practical project, and its `ejb/MarksEntryBean.java` is what `addScore` is copied from. Classroom: `Class.forName` (65), `DriverManager.getConnection(URL, USER, PASSWORD)` (66), `createStatement()` (67), the concatenated `query` (71 to 72), `return st.executeUpdate(query)` (73), `System.out.println(e)` then `return 0` (75 to 76). Constants with the URL and password are on lines 42 to 45. Safe version: `pooled()` (124 to 132), `? placeholders` (84), `ps.setString / setInt` (85 to 88). Also point at `@Remove finished()` (119 to 122), which the servlet calls after each use.

Say: "Same insert, two styles. The classroom one opens a new connection each call, has the password in the source, builds SQL by joining strings and never closes anything. Unlike the practical, it returns how many rows it saved, so the page can tell the truth. The safe one borrows from the pool and binds values."

- 10 **DatabaseConnectivityTest.java**, lines 36 to 58 and 95 to 118. Point at `DriverManager.getConnection(URL, USER, PASSWORD)` (57): a test runs outside the server, so there is no JNDI and no pool. Then `parametersAreNotSql` (97 to 118), which binds `' OR 1=1 --` and asserts zero rows. Say: "Same SQL, same JDBC calls, only the way the connection is obtained differs. The tests skip themselves if MySQL is down."

5. Live demo plan

Set up before you present

MySQL service `MySQL80` running, app running at `http://localhost:8080/placement/`, signed in as `tpo@campus.edu / Campus@2026`. Two windows side by side: the browser and a terminal. Open the mysql client once and leave it open:

```
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user -p placement_cell
(password: Placement@2026)
```

Or use Workbench with the same user. Every mysql query below is a read unless marked.

- 1 **Reports: the live connection.** Go to `http://localhost:8080/placement/admin/reports` (nav link: Reports). The first card, "The live connection", shows Connected and: JNDI name `java:app/jdbc/placementDS`, Database MySQL 8.0.45, JDBC driver MySQL Connector/J 8.4.0, the JDBC URL, Connected as `placement_user@localhost`, Auto commit true, isolation REPEATABLE READ, connection class `com.sun.gjc.spi.jdbc40.ConnectionWrapper40`.

Say: "This is not typed into the page. It is read from DatabaseMetaData on a connection borrowed from the pool. The class is a GlassFish wrapper, not MySQL's own, which is proof it came from the pool."

Prove it in mysql:

```
SELECT CURRENT_USER(), DATABASE(), @@version;
```

- 2 **Reports: the row counts.** Same page, the tiles under the connection card (app_user, student_profile, company, drive, job_application, audit_log). They come from `countRows()`, a plain Statement over a fixed list of table names.

```
SELECT COUNT(*) FROM job_application;
SELECT COUNT(*) FROM student_profile;
SHOW TABLES;
```

The numbers must match the tiles. `SHOW TABLES` lists the same six tables (plus `mock_score` once the classroom page has been used).

- 3 **Reports: aggregate SQL.** Scroll to "Placement by branch". Open "The SQL behind this table". Students per branch today: Computer Science 10, Information Technology 6, Electronics 4, Statistics 2, Mechanical 2, Civil 2.

```
SELECT branch, COUNT(*) AS students FROM student_profile GROUP BY branch;
```

Say: "The report is one SQL statement with joins and aggregates. That is why it is JDBC and not the ORM."

- 4 **Reports: the prepared search.** "Candidate search": Branch Computer Science, Min CGPA 8.5, click the search button. You get Priya Nair 9.3, Manav Shah 8.9, Aarti Deshpande 8.7. The page shows the filter text with `?` placeholders.

```
SELECT s.roll_no, u.full_name, s.cgpa
FROM student_profile s JOIN app_user u ON u.id = s.user_id
WHERE s.branch = 'Computer Science' AND s.cgpa ≥ 8.5
ORDER BY s.cgpa DESC;
```

Same three rows.

- 5 **SQL injection, proved in mysql.** Concatenated versus bound, side by side. Type these one by one:

```
SELECT COUNT(*) AS concatenated FROM student_profile WHERE branch = '' OR '1'='1';

PREPARE s FROM 'SELECT COUNT(*) AS bound FROM student_profile WHERE branch = ?';
SET @b = '' OR '1'='1';
EXECUTE s USING @b;
DEALLOCATE PREPARE s;
```

Today: `concatenated` returns 26 (every student), `bound` returns 0. The first is what string joining would send. The second is what `setString` does: the payload is only ever data.

Do not type the payload into the Reports branch box and say it reached SQL as a literal. See the warning in section 7.

- 6 **Reports: the transaction.** "Close a drive in one transaction". This changes data. Use **Ironbridge Infrastructure : Graduate Engineer Trainee** (drive id 9), which has no applications at the applied stage, so nothing is lost. Click "Close and reject pending" and confirm. The green message says the drive is closed and 0 pending applications were rejected.

```
SELECT id, status FROM drive WHERE id = 9;
SELECT status, COUNT(*) FROM job_application WHERE drive_id = 9 GROUP BY status;
```

Run the first query before and after: OPEN, then CLOSED. Afterwards reopen the drive from the Drives page (the Open action) so the demo data is as it was.

Say: "Two UPDATES, one commit. If the second failed, the first would roll back."

- 7 **Database page: Structure.** Nav: Tools, then Database (/admin/database). In the Tables list on the left click **job_application**, tab Structure. Show the columns, the primary key, the two foreign keys under Relationships, and the unique index **uk_student_drive**.

```
DESCRIBE job_application;
SHOW CREATE TABLE job_application\G
```

Say: "All of this came from DatabaseMetaData: getColumns, getImportedKeys, getIndexInfo. Hibernate created these keys from the JPA mapping. The two views agree."

- 8 **Database page: Rows.** Click the Rows tab, table **app_user**. The **password_hash** column shows (hidden). Then in mysql:

```
SELECT id, email, LEFT(password_hash, 18) AS hash_start FROM app_user LIMIT 2;
```

Say: "The salted PBKDF2 digest is in the table, but the page never returns it. The masking happens in **read()** in the explorer class."

- 9 **Database page: Query console.** Click the Query console tab and run the default query (students by CGPA). Then run the status count below and compare with mysql: 33 REJECTED, 23 APPLIED, 12 SHORTLISTED, 11 INTERVIEW, 6 SELECTED, 85 in all. Then run **DROP TABLE student_profile**: it is refused with "Only SELECT, WITH, SHOW, DESCRIBE and EXPLAIN are allowed here." Try **SELECT 1; SELECT 2**: refused as "Only one statement at a time."

```
SELECT status, COUNT(*) FROM job_application GROUP BY status;
```

Say: "The console only reads: keyword guard, read only connection, 200 row cap, 5 second timeout."

- 10 **Proof of the pool.** In mysql:

```
SHOW PROCESSLIST;
```

You will see a few **placement_user** rows in state Sleep, one per pooled connection (the pool starts at 4), plus your own session. Click through Reports and Database again and run it again: the count does not grow with every page.

Say: "These are the pool's connections. Requests borrow and return them. They are not opened per request."

- 11 **Classroom JDBC page.** Nav: Tools, then Classroom JDBC (/admin/classroom-jdbc). The page shows both code samples and a form. First check the table state in mysql:

```
SHOW TABLES LIKE 'mock_score';
```

Before the first classroom save it returns nothing: **addScore** runs **create table if not exists** itself. Save **Aarti Deshpande** 70, 75, 80 with the method **Classroom (DriverManager and Statement)**. The flash message says it saved and the row appears in "Latest mock scores". Save again with **Pooled (DataSource and PreparedStatement)**. Then in mysql:

```
SELECT * FROM mock_score ORDER BY id DESC;
DESCRIBE mock_score;
SHOW CREATE TABLE mock_score\G
```

Two rows, one from each method.

The next step is the honest injection style demo on the same page.

- 12 Classroom JDBC: what goes wrong with string joining.** On the same page save the name `O'Brien` (scores 70, 75, 80) with the method **Classroom (DriverManager and Statement)**. A red message appears: "The classroom method could not save O'Brien. Its SQL was built by joining strings, and the name changed the statement. The pooled method saves it." No new row shows in "Latest mock scores". The exception is printed to the GlassFish server log (`System.out.println(e)`), line 75): a MySQL syntax error near `Brien` . Now save `O'Brien` again with **Pooled (DataSource and PreparedStatement)**: "Saved O'Brien" and the row appears.

```
SELECT id, sname, aptitude, technical, interview FROM mock_score ORDER BY id DESC;
```

Exactly one `O'Brien` row, the pooled one.

Say: "The apostrophe in the name ended the SQL string early, so the statement changed. That is the everyday face of SQL injection. The bound version treats the same text as plain data."

The servlet checks the name first (letters, spaces and `'` - only), so a full injection payload cannot be typed here. The apostrophe is allowed, and that is enough to show the flaw. Rehearse once. The table `mock_score` exists after any rehearsal; if you want the "table appears" moment on Monday, run `DROP TABLE mock_score;` in mysql first (this table only).

- 13 Optional closer: the tests.** In PowerShell from the project folder run `.\test.ps1` . `DatabaseConnectivityTest` connects to real MySQL with `DriverManager` and checks metadata, the six tables, the injection payload and a rollback. It skips itself if MySQL is down.

6. NetBeans practical vs this project

The practical is the NetBeans project **Prac7-MarksEntry** (`src/java/ejb/MarksEntryBean.java` and `web/index.jsp`). It uses a Stateful bean, the Derby `ClientDriver` , `DriverManager` and a `Statement` . `MockScoreBean` is written in the same shape, pointed at MySQL instead of Derby, and sits beside the real data layer so both can be read side by side. The page `/admin/classroom-jdbc` runs **both** our methods against the same table, `mock_score` , and reads the rows back.

STEP	PRACTICAL: <code>MARKSENTRYBEAN.ADDMARKS</code>	OURS, CLASSROOM: <code>MOCKSCOREBEAN.ADDSCORE</code>	OURS, PROPER: <code>ADDScoresAFELY</code>
Load driver	<code>Class.forName("org.apache.derby.jdbc.ClientDriver")</code> (line 11)	<code>Class.forName("com.mysql.cj.jdbc.Driver")</code> (65)	Not needed: the container owns the driver
Get connection	<code>DriverManager.getConnection("jdbc:derby://localhost:1527/mydb", "root", "root")</code> (12), a new connection every call	<code>DriverManager.getConnection(URL, USER, PASSWORD)</code> (66), a new TCP connection every call	<code>pooled()</code> (124 to 132): JNDI lookup, <code>ds.getConnection()</code> , from the pool
URL and password	Written inside the method	Constants, lines 42 to 45	Nowhere in Java. In <code>glassfish-resources.xml</code>
Statement	<code>con.createStatement()</code> (13)	<code>con.createStatement()</code> (67)	<code>PreparedStatement</code> (83 to 84)
Build the SQL	<code>"insert into marks(sname,marks1,marks2,marks3) values('" + sname + "'," + m1 ...</code> (14 to 15)	Same joining, into <code>mock_score</code> (71 to 72)	<code>values(?,?,?)</code> then <code>setString</code> , <code>setInt</code> (85 to 88)
Run it	<code>st.executeUpdate(query)</code> (16), result thrown away, method is <code>void</code>	<code>return st.executeUpdate(query)</code> (73): returns rows inserted	<code>ps.executeUpdate()</code> (89)
Close	Never: no <code>close()</code> on the statement or the connection	Never: same leak	Try with resources closes the statement and returns the connection
Errors	<code>catch (Exception e) { System.out.println(e); }</code> (17 to 19). <code>index.jsp</code> then prints "Marks entered successfully" whatever happened	Still prints (75), but returns 0 (76). The servlet shows an error message instead of "Saved"	<code>throws SQLException</code> : the servlet shows "The score could not be saved"
Injection	Vulnerable: the name goes straight into the SQL, and the JSP does no validation	Vulnerable by design. A quote in the name changes the SQL. The servlet allows only letters and <code>' -</code> in the name (lines 58 to 59), and <code>O'Brien</code> shows the flaw	Safe: values are data
Who calls it	<code>index.jsp</code> , through a static bean field set in <code>jspInit</code> with a global JNDI lookup	<code>ClassroomJdbcServlet</code> , looking the bean up on each request (line 91) and calling <code>finished()</code> (<code>@Remove</code>) after	Same servlet

How to talk through it

"The practical is the right way to learn the five JDBC steps. It is the wrong way to ship them. Ours does the same insert both ways so you can see what changes: where the connection comes from, how the SQL is built, who closes what, and whether the caller ever learns it failed." Point out that `ClassroomJdbcServlet` validates the name and the 0 to 100 range before either method runs (lines 58 to 61), and looks the bean up by JNDI on every request (line 90) because a Stateful bean is one instance per client and a servlet is shared.

7. Questions you will be asked

CAREFUL

Do not claim the Reports branch box proves injection safety. It is a dropdown, and `ReportsServlet` (line 56) uses `Branches.isKnown`: an unknown value is dropped and the search runs with no branch filter. The protection that matters is the `?` placeholder in the DAO. Prove that with the mysql `PREPARE` demo (step 5) and the test `parametersAreNotSql`. For an in-app example use the O'Brien step on the Classroom JDBC page.

Q What happens if the password in the pool is wrong?

MySQL refuses the login with "Access denied for user placement_user". The password lives in one place, `glassfish-resources.xml` (line 52). The pool is set with `ping="true"`, so GlassFish tests it and reports the failure. In the app, `describeConnection()` catches the `SQLException` (lines 115 to 119) and the Reports card shows "Unavailable" with MySQL's message instead of crashing. Pages that go through Hibernate will also fail. Fix the password in the XML and redeploy with `.\run.ps1`. No Java file changes.

Q What is a connection pool?

A set of already open, already authenticated MySQL connections kept by the server. Code borrows one with `getConnection()` and gives it back with `close()`. Ours (lines 41 to 47): 4 connections opened to start, at most 24, idle ones dropped after 300 seconds, and each is validated before it is handed out.

Q Why not open a connection per request?

Opening a TCP connection and logging in to MySQL costs far more than the query itself, and MySQL allows only a limited number of connections. A pool pays that cost once and reuses. That is exactly what `MockScoreBean.addScore` does wrong: it opens a new connection on every call and never closes it.

Q Is this safe from SQL injection? Show me.

Yes for values. Show `searchCandidates` lines 271 to 299: the SQL only ever gets `?` added, and values are bound with `setObject`. Then run the mysql `PREPARE` demo: concatenated returns 26 rows, bound returns 0. Two honest limits. A table name cannot be a `?`, so `requireKnownTable` (explorer, line 347) accepts only names the database itself reported, and `countRows` (report DAO, 426 to 437) uses a fixed list. And the classroom method is unsafe on purpose, which is why the servlet validates first.

Q What goes wrong with the classroom version?

Four things, all visible in `addScore`. It opens a new connection on every call and never closes it, so connections leak. The URL and password are in the source. The SQL is built by joining strings, so a name like `O'Brien` ends the string early and the statement is rejected, which is SQL injection in its everyday form. And errors are only printed: the practical hides the failure completely, ours returns 0 so the page shows an error, but the exception itself still only goes to the server log. The pooled method saves the same name because the value is bound, not joined.

Q What is the difference between Statement and PreparedStatement?

A Statement sends the finished SQL text. A PreparedStatement sends the SQL with `?` placeholders, and the values travel separately, typed, so they can never be read as SQL. It also lets the database reuse the parsed statement. We still use a plain Statement in a few places where no user value is joined in: row counts over a fixed or database-reported table list, and the console, which runs officer typed reads behind the guard.

Q What if you forget to close the connection?

With a pool, `close()` does not close the socket, it returns the connection. Forget it and the connection is never returned. After 24 leaks the pool is empty and every request waits. That is why every connection here is in a try with resources block, and why `closeDriveAndRejectPending` closes in a `finally`.

Q Why JDBC for reports if you already have Hibernate?

Hibernate is right for loading an object, changing a field and saving it. Reports aggregate across tables and return numbers, never written back. Plain SQL with `COUNT`, `GROUP BY` and a LEFT JOIN is clearer than forcing it into JPQL. Both use the same datasource, so it is one pool, one set of credentials.

Q Explain the transaction in closeDriveAndRejectPending.

Two updates must succeed together: set the drive to CLOSED, and set its APPLIED applications to REJECTED. `setAutoCommit(false)`, run both, `commit()`. On any `SQLException`, `rollback()` undoes both. The finally block restores auto commit, then closes. The bean is `NOT_SUPPORTED` so the container does not start its own JTA transaction, which would conflict with one driven by hand.

Q Can someone destroy data through the SQL console?

Three layers. One: the guard (must start with select, with, show, describe or explain, one statement, no forbidden keyword such as drop, update or into). Two: `setReadOnly(true)` on the connection, so the server refuses writes even if the guard were fooled. Three: the database user only has rights on `placement_cell`. Be honest about layer three: that user does have all privileges on this schema, so layers one and two are what stop writes here. Rows are capped at 200 and a query is cancelled after 5 seconds.

Q Why is the MySQL driver not inside the WAR? Do you still need Class.forName?

The pool is created by GlassFish before the application loads, so the driver must be on the server's classpath: `run.ps1` copies it into `domain1\lib`. And `Class.forName` is not needed with a modern JDBC 4 driver, which registers itself. It appears in `MockScoreBean.addScore` only because the practical uses it.

Q Where is DatabaseMetaData used?

Two places. `describeConnection()` (report DAO, 100 to 121) fills the "live connection" card: product name and version, driver, URL, user, isolation. `describe()` (explorer DAO, 99 to 170) reads columns, primary keys, foreign keys and indexes for the Structure tab. `tableNames()` uses `getTables` as the whitelist.

Q The password is in plain text in an XML file. Is that acceptable?

It is a local database made for this project and the user only has rights on one schema. It is listed as an accepted risk in `docs/SECURITY.md` (item A1). The production step would be a GlassFish password alias or a secret from the environment. The point of the design is that it is in one server file, not compiled into classes.

8. If something goes wrong

SYMPTOM	FIX
Reports card says "Unavailable", or pages show a database error	MySQL is probably stopped. In an admin PowerShell: <code>Start-Service MySQL80</code> . Reload the page. No restart of the app needed.
<code>http://localhost:8080/placement/</code> does not open	The server is down. From the project folder run <code>.\run.ps1</code> and wait for it to deploy. Do not use <code>-Reseed</code> .
Port 8080 already in use	An old server is running. Run <code>.\stop.ps1</code> , then <code>.\run.ps1</code> .
Redirected to the login page	Session expired. Sign in again as <code>tpo@campus.edu</code> . A student account gets a 403 on <code>/admin/</code> pages.
Access denied for user placement_user	Wrong password in <code>glassfish-resources.xml</code> , or the user was not created. Correct password is <code>Placement@2026</code> . Recreate the user with the CREATE USER and GRANT lines in <code>README.md</code> , then <code>.\run.ps1</code> .
<code>mysql</code> is not recognised	Use the full path: <code>& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user -p placement_cell</code> , or use Workbench.
Classroom page shows no scores	Normal. The table <code>mock_score</code> is created by the first classroom method save. <code>SHOW TABLES</code> will confirm.
A POST is rejected (form does nothing or an error page)	The CSRF token went stale. Reload the page and submit again.
Console says a query is refused	Read the reason. It only allows one SELECT, WITH, SHOW, DESCRIBE or EXPLAIN. Words like set, do, use and into are blocked wherever they appear.
Row counts differ from what you rehearsed	Someone applied, closed a drive or saved a score. Re-run your mysql query live; the browser and mysql should always agree with each other.

Written by and for the team. Keep this open in a second window while you present.



TIYA

Accounts, sessions and security

You answer every question about how someone gets in, how they stay in, and what stops the wrong person getting further.

1. Your role in one sentence

You own the front door: creating an account, proving who a person is, keeping them signed in safely, and refusing anyone who is not allowed on a page.

2. What you own

FILE (UNDER <code>SRC/MAIN</code> UNLESS STATED)	LINES	WHAT IT IS
<code>web/LoginServlet.java</code>	181	Sign in form and the sign in decision, session id change, cookies
<code>web/RegisterServlet.java</code>	158	Student sign up: every field checked on the server
<code>web/LogoutServlet.java</code>	48	Ends the session
<code>web/SessionUser.java</code>	69	The small object kept in the session: id, email, name, role. Never the hash
<code>web/SessionTracker.java</code>	43	HttpSessionListener that counts open sessions
<code>web/ThemeServlet.java</code>	46	Light or dark palette kept in a cookie
<code>web/Web.java</code>	107	Shared helpers: forward, redirect, flash messages, CSRF token
<code>security/AuthFilter.java</code>	115	The gate in front of <code>/app/*</code> and <code>/admin/*</code> : login, role, CSRF
<code>security/RequestPaths.java</code>	36	Turns the raw request URL into the path the container routes on, and says whether it is an admin path
<code>security/PasswordHasher.java</code>	97	PBKDF2 hashing and checking, JDK only
<code>security/SecurityHeadersFilter.java</code>	88	CSP and the other protective headers
<code>ejb/AuthService.java</code>	189	The only class that compares a password. Lockout, registration, change password

<code>ejb/AuthResult.java</code>	82	The outcome of a sign in and the wording of each message
<code>util/EmailDomainVerifier.java</code>	262	Disposable domain list and the DNS check on the email domain
<code>webapp/WEB-INF/web.xml</code>	73	Session timeout, cookie flags, error pages, scriptlets off
<code>webapp/WEB-INF/views/login.jsp</code> , <code>register.jsp</code>	138 + 167	The two forms
<code>src/test/.../PasswordHasherTest.java</code> , <code>EmailDomainVerifierTest.java</code> , <code>RequestPathsTest.java</code>	83 + 76 + 49	Tests for hashing, for the email domain checks, and for the admin path rule (5 tests, part of the 146 test suite)

You also lean on `util/Validators.java` (118 lines, shared with Chandra) for the email shape, name and password strength rules.

3. The 60 second explanation

SAY THIS

A student creates an account on the register page. The server checks every field itself, and for the email it does three things: the shape must be valid, the domain must not be a known disposable inbox, and the domain must exist in DNS and be able to receive mail. We do not send a confirmation link, so we prove the domain is real, not that the mailbox is.

The password is never stored. We store a PBKDF2 hash with 120,000 rounds and a fresh random salt for each person, so two people with the same password get different hashes. On sign in we hash what was typed and compare. A wrong password and an unknown email give the same message. Five wrong attempts lock the account for fifteen minutes.

After a good sign in we change the session id, which defeats session fixation, and keep a small user object in the session. Every page under `/app` and `/admin` goes through one filter that checks there is a user, that admin pages need the officer role, and that every POST carries a CSRF token. Every response carries a Content Security Policy, and the JSP files sit under `WEB-INF` so nobody can open them by URL.

4. Code walk in VS Code

Line numbers are from the current files. Open each file, click the line, say the sentence. Do not scroll around.

- 1 `views/register.jsp` , lines 57 to 58, 83, 92. Point at `method="post"` , the hidden `csrf` input, and the two `type="password"` inputs. There is no `value` attribute on either.
Say: "Passwords are typed into masked boxes, sent by POST so they never appear in the URL, and the page is never given a password to print back, so a failed form comes back empty."

- 2** **RegisterServlet.doPost** , lines 64 to 79. Point at the checks in order: CSRF, name, then the email chain (`isEmail` , `isDisposableDomain` , `hasValidMxOrA` , `emailTaken`), then password strength.
Say: "The browser attributes are only a convenience. The server repeats every rule, and the email has four checks that run in this order."
- 3** **Validators.isEmail** , lines 20 to 23 and 34 to 43.
Say: "This is shape only: local part, at, dotted domain, last label two letters or more. `admin@campus` fails here. A made up domain passes here, which is why the next class exists."
- 4** **EmailDomainVerifier.hasValidMxOrA** , lines 140 to 159. Then `lookupViaJndi` (171) and `lookupViaHostname` (235).
Say: "It asks DNS for an MX record, then A and AAAA as RFC 5321 allows. If the lookup cannot be completed at all, for example no network, it lets the sign up through and logs a warning, so a DNS outage cannot lock every student out. If DNS answers that the domain has nothing, it is refused."
- 5** **EmailDomainVerifier.DISPOSABLE_DOMAINS** , lines 77 to 91.
Say: "Forty three well known throwaway providers, such as mailinator. They resolve fine in DNS, so they need their own list."
- 6** **AuthService.registerStudent** , lines 125 to 154. Point at line 129 (email taken again), 132 (roll number clash), 140 to 141 (`PasswordHasher.hash` and `Role.STUDENT`).
Say: "It checks duplicates a second time because two browsers can submit at once. The role is fixed to STUDENT in code. Sign up can never create an officer."
- 7** **PasswordHasher** , lines 22 to 27, 32 to 39, 45 to 63. Point at `PBKDF2WithHmacSHA256` , 120,000 iterations, 16 byte `SecureRandom` salt, the stored format, and `MessageDigest.isEqual` on line 59.
Say: "Stored as `pbkdf2$120000$salt$hash` . The salt is new for every account. The comparison takes constant time. It is JDK only, no library."
- 8** **LoginServlet.doPost** , lines 69 to 121. Read down: CSRF (81), `authService.authenticate` (87), failure path (89 to 93), `changeSessionId` (98), `SessionUser` into the session (103 to 104), cookies (108 to 112), redirect (116 to 120).
Say: "The servlet only decides where to send the person. All password logic is in one bean."
- 9** **AuthService.authenticate** , lines 46 to 92. Point at 52 (bad email shape never reaches the database), 58 to 62 (unknown email still burns a hash), 69 to 72 (locked check), 74 to 85 (the counter), 37 to 38 (the two constants).
Say: "Each wrong password adds one to `failed_attempts` in the database. At five, `locked_until` is set fifteen minutes ahead and the counter resets. While locked, even the right password is refused."
- 10** **AuthResult.badCredentials** , lines 44 to 50.
Say: "Wrong password and unknown email produce the identical sentence, so the form cannot be used to find out who has an account. Once an account has two attempts left, the message adds a countdown."

11 **RequestPaths** , lines 23 to 35, then **AuthFilter** line 53.

Say: "The container routes `//admin/database` to the same servlet as `/admin/database` , but the raw URL still has both slashes. Our first version tested that raw text, so a student could slip past the officer check with a doubled slash. Now `canonical` strips the context path and any semicolon parameter and collapses repeated slashes, and `isAdminPath` tests the cleaned path. Five tests pin it."

12 **AuthFilter** , lines 36, 58 to 61, 63 to 68, 70 to 89.

Say: "One filter guards `/app/*` and `/admin/*` . No user in the session means a redirect to login with the original URL remembered. A student on an admin path gets a 403 and the `error-403.jsp` page. A POST without the right CSRF token is bounced. If reading the token fails because an upload was over the size limit, the user gets a flash message and a redirect, not a 500 page."

13 **Web.csrfToken** and **csrfValid** , lines 73 to 101.

Say: "A 32 byte random token lives in the session and is echoed in a hidden field. A form posted from another site cannot read it."

14 **web.xml** , lines 22 to 31, and **SessionTracker** , lines 22 to 34.

Say: "Session timeout is 30 minutes, the session cookie is HttpOnly, and only cookies are allowed, no session id in the URL. The listener counts sessions created and destroyed, and the officer's System page shows the numbers."

15 **LoginServlet.writeCookie** , lines 171 to 180, and **ThemeServlet** , lines 33 to 37.

Say: "Two cookies on sign in: the email if the box was ticked, and the time of the last visit. Both HttpOnly, 30 days, never a password. The theme cookie is set by the theme servlet for a year and is deliberately readable by scripts."

16 **SecurityHeadersFilter** , lines 51 to 61, 70 to 74, 79 to 84.

Say: "Scripts may only come from our own origin, no inline script at all. Framing is refused, forms may only post to us, and anything that is not a static asset is marked no-store."

17 **Web.render** and **Web.redirect** , lines 44 to 60, then **LogoutServlet** , lines 26 to 38.

Say: "Views are forwarded from `WEB-INF/views` , so a browser cannot ask for them. After a successful POST we redirect. Logout closes the shortlist bean, calls `session.invalidate()` , and redirects to login."

5. Live demo plan: create an account and sign in

About five minutes. Use a private browser window so no old cookies interfere. The app must be running at `http://localhost:8080/placement/` . This is the same demo account the whole presentation uses: Karan hands you the register page already open, and Kinjal, Chandra and Yuvraj continue with this same account after you. Use the exact values below, not made up ones, so it matches the rest of the run sheet: name **Demo Student** , email `demo.student.cs@gmail.com` , password `Demo@2026x` , roll `CS26100` , branch Computer Science, batch year `2026` , CGPA `8.0` , backlogs `0` , phone `9876543210` .

- 1 **Open** `http://localhost:8080/placement/register` . Type anything in a password box and point at the dots. Right click, Inspect, show `type="password"` and `method="post"` .
Say: "The password is masked, and it goes in the request body, not the address bar."

- 2 **Fake email first.** Fill the form: name `Demo Student` , email `demo.student@thisdomaindoesnotexist123.com` , password `Demo@2026x` in both boxes, roll `CS26100` , branch Computer Science, batch year `2026` , CGPA `8.0` , backlogs `0` , phone `9876543210` . Click the create button.
Expect: the form comes back with "That domain doesn't appear to accept email. Check for a typo and try again." beside the email. The two password boxes are empty and the other fields are kept. In the devtools Network tab the response is `400` .
Say: "The address is well formed, so the shape check passes. The domain has no DNS record, so the second check refuses it."
Optional: use `x@mailinator.com` to show "Please use a permanent email address, not a disposable or temporary inbox."

- 3 **Valid sign up.** Change only the email to `demo.student.cs@gmail.com` and type the passwords again. Submit.
Expect: a redirect to the sign in page showing a green panel with a tick, "Welcome, Demo. Your account is ready.", the email and roll number, and three steps with step 1 highlighted. The email is already filled in, the cursor is in the password field, and the button reads "Sign in to your new account". Reload the page and the panel is gone: it is shown once (RegisterServlet puts three values in the session, LoginServlet reads and removes them).
Say: "That was a POST followed by a redirect, so pressing refresh cannot create the account twice."

- 4 **Show the stored password.** Open PowerShell and run:

```
$env:MYSQL_PWD = 'Placement@2026'
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user placement_cell -e "SELECT id, email, user_role, password_hash FROM app_user ORDER BY id DESC LIMIT 3;"
```

Expect: your new row on top with `STUDENT` and a value like `pbkdf2$120000$CX25cv2wd8tNj4Vcb1VCjg==$qBJs55Jt1cqYpZ//gtvCJZPH8ZE+IMaqZmlbD955kYA=` . Point at the four parts: algorithm, 120000 rounds, salt, hash. Password `Demo@2026x` appears nowhere.
Then show the salt matters: add `WHERE email IN ('aarti.deshpande@campus.edu', 'rahul.menon@campus.edu')` . Both seeded accounts share the password `Campus@2026` , yet the salt and hash differ.
Say: "Even we cannot read the password back. A person with the same password gets a different hash because the salt is random per account."

- 5 **Wrong password.** On the sign in page enter `demo.student.cs@gmail.com` and `Wrong@123` .
Expect: "Email or password is incorrect." with HTTP `401` , the email box kept, the password box empty. Repeat with `nobody.here@gmail.com` : the identical sentence.
Say: "We never say which half was wrong. A stranger cannot use this form to find out who has an account."

6 Correct login. Enter `demo.student.cs@gmail.com` and `Demo@2026x`, tick "Remember my email on this device", sign in.

Expect: a redirect to `/placement/app/dashboard` with "Welcome to the placement cell" on the first sign in from a browser, and "Welcome back" after that. In devtools, Application, Cookies:

`JSESSIONID`, `remember_email` and `last_visit`, all HttpOnly, path `/placement`. In Network, open any page and show the `Content-Security-Policy` response header.

Say: "The session id was replaced at this moment. The user object in the session holds the id, email and role, never the hash."

7 Student blocked from admin. Signed in as this student, type

`http://localhost:8080/placement/admin/database` in the address bar.

Expect: the "Not allowed" page, HTTP `403`. Then open a second private window (no session) and visit the same URL: it redirects to the sign in page with "Please sign in to continue." and a `next=` parameter.

Say: "Two different answers: not signed in is a redirect, signed in with the wrong role is a 403."

8 Try to get around it. Still signed in as the student, type

`http://localhost:8080/placement//admin/database` (two slashes after `placement`).

Expect: the same "Not allowed" page, HTTP `403`. `//admin//students` and `/admin;x=1/students` give 403 too.

Say: "We found this one in our own testing. The doubled slash used to get through because the check read the raw URL. Now the check reads the same cleaned path the container routes on."

9 Logout. Click the sign out icon at the top right (its tooltip says "Sign out").

Expect: the sign in page with "You have been signed out." Press the browser Back button: you land on the sign in page, not the dashboard.

Say: "The session is invalidated on the server and every page is sent with no-store, so Back cannot show a cached dashboard."

10 Optional, logout. Sign in to the new account with a wrong password five times. On the third and fourth you see "2 attempts left" and "1 attempt left before this account is locked". The fifth gives "Too many failed attempts. This account is locked for another 15 minutes." A correct password is now refused as well. Undo it: sign in as `tpo@campus.edu`, open Students, open the account, click "Clear the logout".

Never do this on `aarti.deshpande@campus.edu` or the officer account. Use your own demo account.

CAREFUL

After the demo, remove your test account: as the officer open Students, open the account, click "Remove from the register". Do not run any delete query in mysql. In the `locked_until` column the driver stores UTC, so the time can look about five and a half hours behind the clock. That is display only.

6. Servlet course topics mapped to this code

TOPIC	WHERE IN THE CODE
Servlet lifecycle	<code>LoginServlet</code> : the container creates one instance, calls <code>init()</code> once, calls <code>service()</code> for every request (which hands GET to <code>doGet</code> , line 49, and POST to <code>doPost</code> , line 69), and calls <code>destroy()</code> when the app is undeployed. One instance serves every user at once, so the class keeps no per user fields, only injected beans. The same three steps are visible in <code>AuthFilter</code> : <code>init</code> line 42, <code>doFilter</code> line 47, <code>destroy</code> line 112.
Servlet declaration	<code>@WebServlet</code> on <code>LoginServlet</code> line 33. <code>web.xml</code> holds only what has no annotation: session, error pages, JSP settings.
Request parameters, encoding	<code>request.getParameter("email")</code> line 74, <code>setCharacterEncoding("UTF-8")</code> line 72
RequestDispatcher forward	<code>LoginServlet</code> lines 65 and 132, <code>RegisterServlet</code> line 117, <code>AuthFilter</code> line 66, <code>Web.render</code> line 48
Redirect, Post-Redirect-Get	<code>Web.redirect</code> lines 57 to 60, used at <code>LoginServlet</code> line 120 and <code>RegisterServlet</code> line 126. <code>LogoutServlet</code> line 38 and <code>AuthFilter</code> lines 78, 86 and 108 call <code>sendRedirect</code> directly
Request attributes	<code>fail()</code> in <code>LoginServlet</code> , lines 126 to 129: <code>error</code> , <code>email</code> , <code>csrf</code> are read in the JSP as <code>#{error}</code>
HttpSession	Created <code>LoginServlet</code> line 79, user stored line 104, id changed line 98, read in <code>Web.current</code> line 51, destroyed <code>LogoutServlet</code> line 36, timeout <code>web.xml</code> line 23
Cookies	<code>LoginServlet.writeCookie</code> lines 171 to 180, <code>readCookie</code> lines 152 to 169, <code>ThemeServlet</code> lines 33 to 37, session cookie flags in <code>web.xml</code> lines 24 to 27
Filter	<code>AuthFilter</code> (<code>/app/*</code> , <code>/admin/*</code>) and <code>SecurityHeadersFilter</code> (<code>/*</code>), both calling <code>chain.doFilter</code>
Listener	<code>SessionTracker</code> : <code>@WebListener</code> , <code>sessionCreated</code> line 23, <code>sessionDestroyed</code> line 31. Read by <code>SystemService</code> lines 48 and 49
Error pages	<code>web.xml</code> lines 33 to 48: 404, 403, 500 and any uncaught <code>Throwable</code>
JSP, EL, JSTL	<code>login.jsp</code> uses <code>c:out</code> to escape every value and <code>c:if</code> for messages. Scriptlets are switched off in <code>web.xml</code> lines 58 and 67
EJB and injection	<code>AuthService</code> is <code>@Stateless</code> , injected with <code>@EJB</code> at <code>LoginServlet</code> line 39. Every call is logged by the audit interceptor with the password redacted

7. Questions you will be asked

Q How is an account created?

Student self sign up only, at `/register`. `RegisterServlet` checks the CSRF token, name, email (four checks), password strength, roll number, branch, batch year, CGPA, backlogs and phone. Only if there are no errors does it call `AuthService.registerStudent`, which checks for a duplicate email or roll number again, hashes the password, and saves an `app_user` row plus a student profile. The role is hard coded to STUDENT. Officers exist only through the seeder, never through sign up.

Q Would it accept a fake email?

It rejects an address whose domain does not exist: `test@thisdomaindoesnotexist123.com` gets "That domain doesn't appear to accept email". It rejects a malformed one such as `admin@campus`, and 43 known disposable providers such as mailinator. It would still accept an invented mailbox at a real domain, for example `asdf1234@gmail.com`, because we do not check the mailbox.

Q So do you verify the email really exists?

Done: shape check (`Validators.isEmail`), disposable blocklist, and a DNS lookup for an MX record with A and AAAA as fallback (`EmailDomainVerifier`). That proves the domain can receive mail. **Not done:** no confirmation link, no OTP, no test send, and nothing proves the person owns the mailbox. That needs an SMTP account we do not have. It is the recorded next step (Security review S10). If DNS cannot be reached at all, the check lets the sign up through and logs a warning, on purpose.

Q Does it hide the password?

Yes on screen: the inputs are `type="password"` on both forms. The forms use POST, so the password is in the body, not the URL. When a form fails, the servlet forwards the page back with no password value, so the boxes come back empty. The audit trail records the call as `authenticate(email, [redacted])`. Honest limit: the app runs on plain HTTP on localhost, so on a real network it would need HTTPS.

Q How is the password stored? Can you see it?

As `pbkdf2$120000$salt$hash` in `app_user.password_hash`: PBKDF2 with HMAC SHA 256, 120,000 rounds, a 16 byte random salt per account, 256 bit output (`PasswordHasher`). It is one way, so we cannot read the original. Login hashes the typed password with the stored salt and compares in constant time. The salt means two people with the same password get different hashes.

Q What happens after wrong passwords?

Each wrong password adds one to `failed_attempts` in the database (`AuthService` lines 74 to 85). The message is "Email or password is incorrect." with HTTP 401. With two or one attempts left it adds a countdown. The fifth sets `locked_until` fifteen minutes ahead. While locked, even the correct password is refused. A correct sign in resets the counter. The officer can clear a lockout early on the student's page. The lock lives in the database, so restarting the server does not clear it, and it counts per account, not per computer.

Q Can someone tell which emails have accounts?

Not from the wrong password message: unknown email and wrong password read the same, and an unknown email still burns one hash so the timing matches. Two small leaks exist and you should say so: the countdown and lockout messages only appear for real accounts, and the register page says "An account already exists for that address".

Q Can a student open the admin pages?

No. `AuthFilter` line 63 checks the role for every path under `/admin`, and a student gets HTTP 403 with the "Not allowed" page. Someone not signed in is redirected to login. The path it tests is cleaned first by `RequestPaths.canonical`, so `//admin/database` and `/admin;x=1/students` are refused too. We tested all of these as a student. This was a real bug in an earlier version: the check read the raw URL, the container routed on the cleaned one, and a doubled slash got through. It is fixed and covered by `RequestPathsTest`.

Q What is session fixation and how do you stop it?

An attacker plants a known session id in the victim's browser, waits for the victim to sign in, then uses the same id. We call `request.changeSessionId()` right after a successful password check (`LoginServlet` line 98), so the pre login id becomes useless. Only cookies are allowed for the id (`web.xml` line 30), so it cannot sit in a URL.

Q How long does a session last, and who counts them?

30 minutes of inactivity: `web.xml` line 23, and `SessionTracker` line 24 sets the same 30 minutes when each session is created. Logout invalidates it at once. `SessionTracker` implements `HttpSessionListener`; the container calls it on creation and on invalidation or timeout, and the officer's System page shows "Sessions open now".

Q Which cookies do you set, and with what flags?

`JSESSIONID`: `HttpOnly` (in `web.xml`). `remember_email` (only if the box is ticked) and `last_visit`: `HttpOnly`, 30 days, path `/placement`, values percent encoded. `portal_theme`: one year, not `HttpOnly`, only says light or dark. No cookie holds a password. Not set: the `Secure` flag, because a secure cookie would never be sent over plain HTTP on localhost (accepted risk A2 in the review), and no `SameSite` attribute is set in our code.

Q Why are the JSPs under WEB-INF? Forward or redirect?

Anything under `WEB-INF` cannot be requested by URL: we tested `/placement/WEB-INF/views/login.jsp` and got 404. Only a servlet can reach a view, by `RequestDispatcher.forward`, which happens inside the server: the address bar keeps the servlet URL. A redirect sends the browser a 302 and it makes a second request. We forward to show a page, including a failed login, and redirect after every successful POST (Post-Redirect-Get) so refresh cannot repeat the action.

Q What are CSRF and the Content Security Policy?

CSRF: another site making your browser submit a form to us. Every POST needs the random per session token in a hidden field (`Web.csrfValid`). **CSP:** `script-src 'self'` with no inline scripts, so the browser refuses any script we did not serve. Plus `frame-ancestors 'none'` and `X-Frame-Options: DENY` against clickjacking, `form-action 'self'` , `nosniff` , and no-store on every page that is not a static asset. Not set: HSTS, because we have no HTTPS.

8. If something goes wrong

Fixed: the admin role check bypass

What it was. The officer check tested the raw request URL with `startsWith("/admin/")` . The container routes on a cleaned path, so `/placement//admin/database` reached the officer Database page as a student. **Why.** The raw URL and the routed path are not the same text. **The fix.** `RequestPaths.canonical` (strip context path and `;` parameters, collapse repeated slashes) and `RequestPaths.isAdminPath` , used at `AuthFilter` lines 53 and 63. **Checked.** As a student, `/admin/database` , `//admin/database` , `//admin//students` and `/admin;x=1/students` all return 403. If asked, say plainly that we found it ourselves and closed it.

WHAT YOU SEE	WHY, AND WHAT TO DO
Sign up says the domain "doesn't appear to accept email" for a real address	DNS on this network answered that the name does not exist. Try another network, or use <code>gmail.com</code> . If DNS cannot be reached at all the code lets the sign up through, so a refusal means a definite "no".
"This page sat open too long to be verified" or "could not be verified"	The CSRF token did not match: the session expired (30 minutes), cookies are blocked, or the page was opened in another window. Reload the page and fill it in again.
"An account already exists" or "Roll number ... is already registered"	You already used that email or roll number in a rehearsal. Pick new ones, or remove the old test account as the officer.
Demo account is locked	Five wrong attempts. Wait 15 minutes, or as the officer open Students, open the account, click "Clear the logout".
Correct password refused with 401	Check whether the message says locked, deactivated, or "not a valid email address". Each has its own text. Otherwise check Caps Lock: the email is not case sensitive, the password is.
Signed out in the middle of the demo	30 minute inactivity timeout, or the server was restarted. Sign in again.
mysql says "Access denied ... using password: NO"	Set <code>\$env:MYSQL_PWD = 'PLacement@2026'</code> in the same PowerShell window first.
The app does not load	Do not restart it yourself mid presentation. Ask whoever started it to check the server window.

What is not implemented (say it plainly if asked)

- No email confirmation link, no OTP, no forgot password flow. A student who forgets a password cannot reset it by email. The change password action needs the current password (`AuthService.changePassword`).
- Only the email domain is checked. The mailbox is not.
- No HTTPS, so no `Secure` cookie flag and no HSTS. On a real network the password and session id would travel in clear text.
- Lockout is per account, not per address, and the seeded demo accounts share one password (accepted risk A6 in the security review).
- Logout works on a plain GET link, so another site could sign a user out. That is a nuisance, not a data risk.
- The roles are two only: officer and student. There is no role management screen.



KINJAL

Kinjal: Views and Validation

Campus Placement and Training Cell. Everything a person sees, types or uploads. Every line number below was checked against the current source.

1. Your role in one sentence

You own the last step of every request: the JSP pages that turn data into HTML, the rules that decide whether typed input is acceptable, and the two ways a file gets in (the resume upload and the bulk import).

2. What you own

FILE	LINES	WHAT IT IS
layout-top.jspf	146	Shared page opening: theme cookie, header, navigation, flash messages.
layout-bottom.jspf	16	Shared footer, with the one <code>jsp:useBean</code> and one <code>fmt</code> tag.
icons.jspf	48	Hidden SVG sprite holding every icon.
Student pages: <code>student-dashboard</code> 191, <code>drives</code> 106, <code>drive-detail</code> 150, <code>my-applications</code> 76, <code>profile</code> 186	709	What a student sees. <code>profile.jsp</code> holds the details form, password form and resume upload.
Officer pages: <code>admin-dashboard</code> 106, <code>admin-students</code> 88, <code>admin-student-detail</code> 140, <code>admin-companies</code> 131, <code>admin-drives</code> 224, <code>admin-applications</code> 107, <code>admin-shortlist</code> 133, <code>admin-audit</code> 70, <code>admin-system</code> 76	1075	What the placement officer sees.
admin-import.jsp	81	The bulk import page (a text box and one button).
<code>error-403</code> 34, <code>error-404</code> 30, <code>error-500</code> 40	104	Error pages wired in <code>web.xml</code> .
<code>assets/app.css</code> 510, <code>app2.css</code> 392	902	All styling, light and dark palettes as colour tokens.

<code>assets/app.js</code>	136	The only script in the app: confirm dialogs and the import button.
<code>assets/favicon.svg</code> 11, <code>assets/fonts/</code> (four Plus Jakarta Sans woff2 files)	11	Icon and self-hosted fonts, so nothing loads from another site.
<code>util/Validators.java</code>	118	Server side input rules (email, roll number, phone, name, password, safe number parsing).
<code>util/Branches.java</code>	36	The fixed list of six branches.
<code>util/UploadStore.java</code>	138	Where resumes live, safe names, file type checks.
<code>web/ProfileServlet.java</code>	236	Profile page: details, password change, resume upload.
<code>web/ResumeDownloadServlet.java</code>	80	Serves a stored resume back, with an owner check.
<code>web/BulkImportServlet.java</code>	218	Non blocking bulk import of students.
<code>ValidatorsTest</code> 153, <code>BranchesTest</code> 50, <code>UploadStoreTest</code> 93	296	JUnit 5 tests for your three util classes.

Not yours: `login.jsp`, `register.jsp`, `admin-reports.jsp`, `admin-database.jsp`, `admin-classroom.jsp`. You will still be asked about the register page, so know it. Also note `webapp/index.jsp` (18 lines): the welcome file, the only JSP outside WEB-INF, no markup in it.

3. The 60 second explanation

SAY THIS

My part is everything the user touches. Pages are JSP with JSTL and EL only. Scriptlets are switched off in web.xml, so no Java can be written into a page. A servlet puts its data on the request and forwards to a JSP under WEB-INF, which is why no page can be opened by URL. Every page includes the same header and footer with the include directive, and user supplied text is printed with `c:out` so it is escaped.

Input is checked twice. The forms carry HTML attributes as hints, but the server decides, using the Validators class, and shows the error next to the field. All JavaScript is in one file, app.js, because our security policy forbids inline scripts. I also own two file paths in: the resume upload, limited by size, extension and the first bytes of the file, stored under a generated name and served back through a servlet; and the bulk import, which reads the request without holding a thread, using a ReadListener.

4. Code walk in VS Code

- 1 **WEB-INF/web.xml , lines 50 to 70.** Point at the two `jsp-property-group` blocks and the line `<scripting-invalid>true</scripting-invalid>` (lines 58 and 67, for `*.jsp` and `*.jspx`). Say: "Scriptlets are refused by the container. A page with Java in it would not even compile, so every page has to use EL and JSTL."
- 2 **web/Web.java , lines 32 and 44 to 49.** Point at `VIEW_ROOT = "/WEB-INF/views/"` and `getRequestDispatcher( ... ).forward( ... )`. Say: "Every servlet ends with this call. The JSPs sit under WEB-INF, so the container will not serve them directly. Only a forward from inside the server can reach them."
- 3 **DriveBrowseServlet.java , lines 55, 65 and 70, then drives.jsp .** Point at `request.setAttribute("drives", drives)`, then `<c:forEach var="d" items="${drives}">` at line 33 of the JSP. Say: "The servlet puts the list on the request. The JSP reads it back by name with EL. That is the whole handover."
- 4 **layout-top.jspf , lines 11 to 13, 25 and 29.** Point at the three `c:set` lines (`ctx` , `me` , `theme`), the `cookie.portal_theme` expression, and `<html lang="en" data-theme="${theme}">`. Say: "The theme comes out of a cookie through the `cookie` implicit object. The page title is printed with `c:out`."
- 5 **drives.jsp line 4 and last line, layout-bottom.jspf line 6.** Point at `<%@ include file="layout-top.jspf" %>`, then `jsp:useBean` and the `fmt:formatDate` on line 12. Say: "The include directive pastes the fragment in at translation time, so the variables it sets are visible to the page. `jsp:useBean` is the one standard action we use."
- 6 **layout-top.jspf , lines 133 to 145.** Point at the flash message block and `<c:remove var="flashSuccess" scope="session"/>`. Say: "A message is shown once and removed, so a refresh does not repeat it."
- 7 **drives.jsp , lines 38 to 40 and 73 to 97.** Point at `<c:out value="${d.company.name}"/>` and the `c:choose` with `c:when` and `c:otherwise` for Already applied, Eligible, Not eligible. Say: "Text comes from the database, so it is always escaped. The verdict was decided in the EJB, the page only shows it."
- 8 **profile.jsp lines 19 to 40, ProfileServlet.java lines 91 to 121.** Point at `novalidate` , `required` , `min` , `max` , `aria-invalid` , the `div class="err"` ; then the `errors` map in the servlet. Say: "The attributes are hints. The servlet runs Validators, collects messages in a map, and forwards back to the same page, where each message sits under its field."
- 9 **util/Validators.java lines 6 to 10, 20 to 29, 61.** Point at the class comment, the `EMAIL` pattern, and `isStrongPassword`. Say: "Deliberately stricter than the RFC. `admin@campus` is refused. The HTML attributes are not a security control, this class is." Then run `.\test.ps1` if asked: 27 test methods across your three test classes.

10 `assets/app.js` lines 1 to 31, `security/SecurityHeadersFilter.java` lines 51 to 61. Point at the header comment and the `submit` listener that reads `data-confirm`, then `script-src 'self'` in the CSP. Say: "The browser runs only scripts served from our own origin. So no page may contain a script block or an onclick. The confirm dialogs use a data attribute and one listener."

11 **Upload path.** `profile.jsp` lines 154 to 165 (`enctype="multipart/form-data"` , `accept`), `ProfileServlet.java` lines 32 to 35 (`@MultipartConfig` , 2 MB file, 3 MB request), 73 to 82 (an oversize file is caught here) and 159 to 214, `UploadStore.java` lines 36 to 39 and 92 to 107, and `security/AuthFilter.java` lines 70 to 80 (the same oversize catch, because the filter reads the token first). Say: "Size first, then extension, then the opening bytes must match the format, then the file is written under a name we generate."

12 `ResumeDownloadServlet.java` lines 41 to 78. Point at the owner check, `UploadStore.resolve` , `Content-Disposition: attachment` , `nosniff` . Say: "The file is found from the student record, never from a name in the URL. A student gets 403 for anyone else's resume."

13 **Bulk import.** `BulkImportServlet.java` line 40 (`asyncSupported = true`), line 67 (`startAsync`), lines 73 to 118 (`ReadListener` : `onDataAvailable` , `onAllDataRead` , `onError`); `admin-import.jsp` lines 36 to 39; `app.js` lines 109 to 135. Say: "The button posts the pasted text with fetch. The servlet reads it in chunks only when the container says bytes are ready."

14 `assets/app.css` line 102 and `web/ThemeServlet.java` . Point at `[data-theme="dark"]` replacing the colour tokens. Say: "Dark mode is a token swap, not a second stylesheet."

5. Live demo plan

Prepare before the demo: one small real PDF (under 2 MB) on the desktop, one text file renamed to `fake.pdf` , and one file over 2 MB (any large file, for example a video or a big zip). When this was written nobody had a resume on file, so the upload has to come first.

1 **Direct JSP is refused.** Open `http://localhost:8080/placement/WEB-INF/views/drives.jsp` . You get the "There is nothing at that address" page (a 404, not 403). Say: "The container hides WEB-INF. The only way to that file is a forward."

2 **Sign in as the student.** `http://localhost:8080/placement/login` , `aarti.deshpande@campus.edu` , `Campus@2026` . You land on `/app/dashboard` . Click the moon icon at top right: the address flashes `/theme?mode=dark&back= ...` and returns dark. Click the sun to go back. Say: "A cookie called `portal_theme`, read with EL, sets `data-theme` on the html tag."

- 3 **A JSP showing database data.** Click **Drives** (`/placement/app/drives`). You see 10 cards, first **Saffron Retail Tech**. Right click, View page source: one header, escaped text, no script tags except the app.js link. Then say the path: browser, `/app/drives` , `AuthFilter` , `DriveBrowseServlet.doGet` , `DriveService.listOpen()` (a stateless EJB running the `Drive.open` named query), `request.setAttribute("drives")` , `Web.render` , forward to `/WEB-INF/views/drives.jsp` , `c:forEach` , `c:out` . Optional proof: run `select c.name from drive d join company c on c.id=d.company_id where d.status='OPEN' order by d.last_date` in MySQL and the order matches.
- 4 **A validation error.** Click **Profile** (`/app/profile`). Set CGPA to `99` and Mobile number to `12` , click **Save details**. The page returns with red text under each field: "CGPA must be between 0 and 10." and "Enter a ten digit mobile number." Nothing is saved, and the boxes show the saved values again. Say: "The browser did not stop me, the form has novalidate. The server decided."
- 5 **Escaping.** Open a private window on `http://localhost:8080/placement/register` . Full name `"><b>hi</b></code> , email admin@campus , leave the rest bad, click Create account. The text comes back inside the name box exactly as typed, not as bold. View source shows "><b> . No account is created. Say: "This is c:out."`
- 6 **Resume upload and download.** Back on the student **Profile**, right hand card. Choose `fake.pdf` , click **Upload**: red banner "That file does not look like a .pdf document inside, whatever it is named." Now choose the real PDF, click **Upload**: green banner "Resume uploaded." and the card says a resume is on file. Click **Download the copy on file**: the browser saves `Resume-CS21001.pdf` . Say where it lives: `domains/domain1/placement-resumes/resume-1-xxxxxxx.pdf` , outside the deployed app. This changes data: Aarti now has a resume on file.
- 7 **Oversize file.** On the student **Profile** choose the file over 2 MB and click **Upload**. You return to the profile with the red banner "That file is larger than the two megabyte limit." Nothing is stored. Say: "The container refuses it, and we turn that into a normal message instead of an error page." For the real upload keep using the small PDF.
- 8 **Another student's resume is refused.** Type `http://localhost:8080/placement/app/resume?student=2` while signed in as Aarti. You get the 403 page. Student 2 has no resume, so that is the refusal to show. As the officer, **Students**, Aarti Deshpande, the Resume row shows **Download** (`?student=1`) and it works.
- 9 **Bulk import as officer.** Sign out, sign in as `tpo@campus.edu` , `Campus@2026` . Open **Tools**, then **Import** (`/placement/admin/import`). Click **Import these rows** with the pre filled text. Today's result: 1 created (`IT22011` , Karan Bhatt), 2 refused: row 1 "An account already exists for priya.nair@campus.edu" (she is already seeded) and row 3 "broken-address is not a valid email address". Click **Open the student register** to show Karan Bhatt. This creates a real account: remove it afterwards from **Students**, open Karan Bhatt, **Remove from the register** (that button uses the data-confirm dialog from app.js).

- 10 Role guard on a page.** Optional, as the student: `http://localhost:8080/placement/admin/import` shows the 403 page rendered from `error-403.jsp`.

CAREFUL

This is a different, separate fake student from the "Demo Student" the rest of the presentation uses, so it does not collide with that account. Rehearse the import with the first two rows changed to a new fake name, email and roll number (for example `Priya Patil,priya.patil.import@campus.edu,CS22099,Computer Science,2027,7.5,0,9820100299`) so the real demo still has something to create. A second run of the same rows creates nothing.

6. Servlet and JSP course topics mapped to this code

TOPIC	WHERE IN THE CODE
Page directive	Line 1 of every view: <code><%@ page contentType="text/html; charset=UTF-8" %></code> . <code>error-500.jsp</code> adds <code>isErrorPage="true"</code> .
Taglib directive	<code><%@ taglib prefix="c" uri="jakarta.tags.core" %></code> on every page, plus <code>fmt</code> in <code>layout-top.jspf</code> .
Include directive	Every page: <code><%@ include file="layout-top.jspf" %></code> first, <code>layout-bottom.jspf</code> last. <code>layout-top.jspf</code> includes <code>icons.jspf</code> .
Standard action	<code>jsp:useBean</code> in <code>layout-bottom.jspf</code> line 6. This is the only one. <code>jsp:include</code> and <code>jsp:forward</code> are not used.
JSTL core	<code>c:set</code> , <code>c:if</code> , <code>c:choose/when/otherwise</code> , <code>c:forEach</code> , <code>c:out</code> , <code>c:remove</code> in the views; <code>c:redirect</code> in <code>index.jsp</code> .
JSTL formatting	<code>fmt:formatDate</code> in <code>layout-bottom.jspf</code> line 12 (the render time in the footer).
EL implicit objects	<code>pageContext</code> (<code>pageContext.request.contextPath</code> , <code>pageContext.exception</code>), <code>sessionScope</code> (<code>sessionScope.user</code> , flash messages), <code>requestScope</code> (the forward path, <code>layout-top.jspf</code> line 23), <code>cookie</code> (<code>cookie.portal_theme</code>). <code>param</code> and <code>header</code> are not used.
EL operators	<code>empty</code> , <code>not</code> , <code>? :</code> , <code>==</code> , <code>eq</code> , map lookup <code>verdicts[d.id]</code> , method call <code>applications.size()</code> .
Scopes	Page: <code>ctx</code> , <code>me</code> , <code>theme</code> . Request: <code>pageTitle</code> , <code>drives</code> , <code>detailErrors</code> . Session: <code>user</code> , <code>flashSuccess</code> , <code>csrfToken</code> .
Servlet to JSP	<code>request.setAttribute</code> then <code>Web.render</code> , which calls <code>RequestDispatcher.forward</code> (<code>Web.java</code> lines 44 to 49).
No scriptlets	<code>web.xml</code> <code>scripting-invalid</code> , lines 58 and 67.
Error pages	<code>web.xml</code> lines 33 to 48: 404, 403, 500 and <code>java.lang.Throwable</code> mapped to the three JSPs.
Redirect after POST, flash	<code>Web.redirect</code> , <code>Web.flashSuccess</code> ; <code>ProfileServlet</code> ends every save with a redirect.

Cookies	Written by <code>ThemeServlet</code> , read in <code>layout-top.jspf</code> line 13.
File upload	<code>@MultipartConfig</code> , <code>Part</code> , <code>getSubmittedFileName</code> in <code>ProfileServlet</code> .
File download	<code>ResumeDownloadServlet</code> : content type, <code>Content-Disposition</code> , streaming with <code>transferTo</code> .
Async and non blocking I/O	<code>startAsync</code> , <code>ReadListener</code> , <code>AsyncContext.complete</code> in <code>BulkImportServlet</code> .
Input validation	<code>Validators</code> , <code>Branches</code> , used by <code>ProfileServlet</code> and <code>BulkImportServlet</code> (and by register).

7. Questions you will be asked

Q Why can't I open your JSPs directly?

They live under `WEB-INF/views` , and the container never serves anything under WEB-INF to a browser. The only way in is `RequestDispatcher.forward` from a servlet, which is what `Web.render` does. Try it: `/placement/WEB-INF/views/drives.jsp` gives a 404. The one exception is `index.jsp` , the welcome file, and it holds no markup.

Q Why are scriptlets disabled, and where?

In `web.xml` , `<scripting-invalid>true</scripting-invalid>` for `*.jsp` and `*.jspx` . Java inside a page mixes logic into the view, cannot be tested, and is where injection bugs hide. With it off, a page containing a scriptlet fails to compile, so the rule cannot be broken by accident.

Q Include directive or `jsp:include` ? Which do you use?

The include directive, `<%@ include file="layout-top.jspf" %>` . It is a static include: the fragment is pasted in when the page is translated, so the whole page becomes one servlet and variables like `ctx` and `me` set in the fragment are visible to the page. `jsp:include` would run at request time as a separate resource. We do not use it.

Q What does `c:out` do, and why is it on every user value?

It escapes `&` `<` `>` `"` `'` before printing. Without it, a name like `"><script>alert(1)</script>` would be written into the page as live markup. With it, the register page sends back `"><script>alert(1)</script>` , which the browser shows as plain text. Plain EL like `${x}` does not escape. I checked this against the running register page.

It is used inside attributes too, not only in page text. The `data-confirm` messages that contain a name (`admin-companies.jsp` line 52, `admin-student-detail.jsp` lines 101 and 129, `my-applications.jsp` line 52) use `<c:out value='${co.name}' />` inside the attribute. Plain EL remains only for numbers and server made values such as ids, the csrf token and status labels. One client supplied exception: the theme cookie value on the html tag, which only the person's own browser can set.

Q How does a servlet hand data to a JSP?

It calls `request.setAttribute("drives", drives)` and then forwards. The JSP reads `${drives}`. EL searches page, request, session, then application scope for the name. The address bar keeps the servlet URL because a forward happens inside the server.

Q Forward or redirect, and when?

Forward to show a page: same request, attributes survive, no second round trip. Redirect after a successful POST so a refresh does not repeat the write. A message that must survive the redirect goes in the session as `flashSuccess` or `flashError`, and `layout-top.jspf` removes it with `c:remove` after printing.

Q Is validation done in the browser or on the server?

Both, but only the server counts. Inputs carry `required`, `min`, `max`, `maxlength`, `type="email"`, and the forms also carry `novalidate`, so the browser does not block a submit or show its own bubbles. `Validators` does the real check and the page shows the message under the field with `aria-invalid="true"`. The register page says it plainly: "Every field is checked again on the server before anything is saved." Anyone can bypass a browser with curl.

Q Why is there no inline JavaScript?

The Content-Security-Policy header says `script-src 'self'` with no `'unsafe-inline'`, so the browser runs only scripts served from our own origin. An injected script tag would sit there unused. That only works if no page has a script block or an onclick, so everything is in `app.js`. Confirm dialogs use a `data-confirm` attribute and one listener. Inline *styles* are allowed by the policy and a few pages use them.

Q How does light and dark mode work?

Clicking the icon calls `/theme?mode=dark`. `ThemeServlet` stores `portal_theme` in a one year cookie, accepting only `light` or `dark`. Each page reads it with `cookie.portal_theme.value` and puts it on the html tag as `data-theme`. `app.css` line 102 swaps the colour tokens under `[data-theme="dark"]`. It does not follow the operating system setting.

Q How is the resume upload kept safe?

Four checks and one placement rule (plus a friendly message for an oversize file). `@MultipartConfig` caps a file at 2 MB and a request at 3 MB. The extension must be pdf, doc or docx. The first bytes must match that format (`%PDF`, or the zip or OLE signature). An oversize file is caught and answered with a normal message (`ProfileServlet` lines 73 to 82, `AuthFilter` lines 70 to 80). The saved name is generated (`resume-1-a1b2c3d4.pdf`), so the browser's file name is never used as a path. The file goes to `domain1/placement-resumes`, outside the deployed app, so a redeploy does not wipe it.

Q How is a resume served back, and can a student read someone else's?

Through `/app/resume`, not a static link. The servlet finds the file name from the student record, resolves it inside the upload folder only, and sends it with `Content-Disposition: attachment` and `nosniff`. A student asking for another student's id gets 403. The officer may fetch any.

Q Explain the non blocking bulk import in plain words.

Normally `read()` makes a server thread wait for the next byte, so a slow upload ties up a thread doing nothing. In `BulkImportServlet` the request is put in async mode (line 67) and a `ReadListener` is attached (line 73). The container calls `onDataAvailable` only when bytes are ready, we read while `isReady()` is true, then the thread goes back to the pool. When the body ends, `onAllDataRead` validates the rows, writes JSON and calls `async.complete()`. Limits: 512 KB and a 30 second timeout. The non blocking part is reading the request. The database work for each row still runs on that callback thread.

Q Is the import a file upload?

No. The officer pastes text into a box and `app.js` sends it with `fetch` as `text/csv`. It is not multipart, which is why it is a different mechanism from the resume upload. The CSRF token goes in the query string because there is no form body to carry it. The script writes the JSON reply into the page with `textContent`, never `innerHTML`.

Q What accessibility work did you do?

What is in the markup: `lang="en"` on every page; a `label for` on form fields; `aria-invalid` set by the server with visible error text, so colour is not the only signal; `aria-hidden` on the logo and icon sprite; `aria-label="Menu"` on the menu toggle; a visible `:focus-visible` outline, including on the CSS only menus; transitions only when the user has not asked for reduced motion; and a written contrast audit in the comments of `app.css`. Not done: no skip link, and the theme and sign out icons have `title` text but no `aria-label`.

8. If something goes wrong

WHAT YOU SEE	WHY, AND WHAT TO DO
Red banner "That file is larger than the two megabyte limit."	The file is over 2 MB. That is the intended message. Use the small PDF for the real upload.
Officer opens <code>/app/profile</code> and lands on the dashboard	Expected. An officer has no profile page, so <code>ProfileServlet</code> redirects to <code>/admin/dashboard</code> . Show the upload as the student.
Upload refused	The name must end in <code>.pdf</code> , <code>.doc</code> or <code>.docx</code> and the content must match. A renamed text file is refused on purpose.
Import says "The import could not be completed"	Session expired, so the request was bounced to the login page. Sign in again, reopen Import.

Import creates nothing	Those emails or roll numbers already exist, for example a second run. Change the email and roll number in the rows.
Download says no resume	Nothing has been uploaded for that student yet (nobody has one at present). Upload first.
Theme does not switch	Cookies blocked in this browser window. Use a normal window, then hard refresh.
Page unstyled or icons missing	Hard refresh (Ctrl+F5). Check <code>/placement/assets/app.css</code> returns 200.
Every page fails, connection refused	The server is down. Tell the team member who runs it. Do not start it yourself mid demo.



CHANDRA

Chandra: Entities and EJB

Domain model, Hibernate, the session beans, the queue and the audit trail.
Campus Placement and Training Cell.

1. Your role in one sentence

You own the domain model and the business logic: the six JPA entities that Hibernate maps onto MySQL, and the EJBs that read and write them, decide who is eligible, queue notifications and record the audit trail.

2. What you own

FILE	LINES	WHAT IT IS
entity/AppUser.java	158	Account and credentials
entity/StudentProfile.java	155	Academic record, one to one
entity/Company.java	105	Recruiter, @OneToMany to drives
entity/Drive.java	207	Drive and its eligibility inputs
entity/JobApplication.java	133	Student to drive join, unique key
entity/AuditLog.java	102	One audit row
entity/Role, ApplicationStatus, DriveStatus	20, 28, 22	The three enums
META-INF/persistence.xml	68	Persistence unit, Hibernate provider
ejb/ApplicationService.java	171	Stateless: apply, status, withdraw
ejb/DriveService.java	153	Stateless: drives, companies, check()
ejb/StudentService.java	109	Stateless: profiles, search
ejb/EligibilityRules, Eligibility	69, 46	The rules, and the verdict object
ejb/ShortlistBean.java	136	Stateful shortlist basket
ejb/NotificationSender.java	56	JMS producer, declares the queue
ejb/NotificationMDB.java	58	Message driven consumer
ejb/PortalStatsBean.java	157	Singleton: counters, JNDI report

FILE	LINES	WHAT IT IS
<code>ejb/AuditInterceptor.java</code>	161	<code>@AroundInvoke</code> audit interceptor
<code>ejb/AuditWriter, AuditService</code>	30, 42	Write one row, read the rows
<code>test/ejb/EligibilityRulesTest.java</code>	179	10 tests, no database

Not yours: `AuthService` and `AuthResult` (Tiya), `DataSeeder` (a teammate), `MockScoreBean` (Yuvraj, also stateful). `EligibilityRules` and `Eligibility` are plain classes, not beans. `ShortlistServlet` is Karan's but drives your `ShortlistBean`. Your interceptor also wraps `AuthService`, which is why sign ins show on the Audit page.

3. The 60 second explanation

SAY THIS

My part is the model and the business rules. Six JPA entities, mapped by Hibernate onto MySQL. An application row joins a student to a drive, with a unique key on the pair so nobody can apply twice.

The rules live in EJBs. The three services are stateless and the container wraps each write in a transaction. The shortlist basket is stateful, one per officer. A singleton starts at deploy. When an application changes, a message goes on a JMS queue and a message driven bean records it. An interceptor writes an audit row for every change, with passwords redacted. Eligibility is one plain method that lists every reason a student fails, so I can show it live.

4. An EJB cheat table

BEAN	TYPE	ANNOTATIONS	PURPOSE	USED BY
<code>ApplicationService</code>	Stateless	<code>@Stateless @Interceptors</code>	Apply, status, withdraw	Student, officer servlets
<code>DriveService</code>	Stateless	same	Drives, <code>check()</code>	Drive pages
<code>StudentService</code>	Stateless	same	Profiles, search	Profile pages
<code>NotificationSender</code>	Stateless	<code>@JMSDestinationDefinition</code>	Queue a message	Services, basket

BEAN	TYPE	ANNOTATIONS	PURPOSE	USED BY
<code>NotificationMDB</code>	Message driven	<code>@MessageDriven</code>	Consume, save a row	Container only
<code>ShortlistBean</code>	Stateful	<code>@Stateful @Remove</code>	Officer's basket	<code>ShortlistServlet</code>
<code>PortalStatsBean</code>	Singleton	<code>@Singleton @Startup</code>	Counters, JNDI	Login, apply, System

BEAN	TYPE	ANNOTATIONS	PURPOSE	USED BY
<code>AuditInterceptor</code>	Interceptor	<code>@AroundInvoke</code>	Audit row per call	Three services, <code>AuthService</code>

<code>AuditWriter</code>	Stateless	<code>REQUIRES_NEW</code>	Save audit row	The interceptor
<code>AuditService</code>	Stateless	<code>SUPPORTS</code>	Read audit rows	Audit page

Transactions and rollback

- `SUPPORTS` : every read in the three services and all of `AuditService` . Join a transaction if there is one, never start one.
- `REQUIRED` (default, no annotation): every write (`apply` , `updateStatus` , `withdraw` , `save` , `delete` , `ShortlistBean.commit`) and the MDB. Written out on `NotificationSender.send` .
- `REQUIRES_NEW` : `AuditWriter.write` , so the audit row commits even if the caller rolls back. MANDATORY and NEVER are not used, and there is no `UserTransaction` anywhere.

What a rollback means. The container starts the transaction before a business method and commits when it returns. A runtime exception is a system exception: the transaction rolls back, so the INSERT or UPDATE from that call never happens, and the caller receives the original wrapped in `EJBException` . That is why `ApplicationServlet.withdraw` catches `EJBException` and reads `getCause()` . `ShortlistBean.commit()` therefore moves every ticked candidate or none. JMS errors are swallowed on purpose, so a broken queue never undoes an edit.

JNDI, and @EJB compared with a lookup

- `java:app/jdbc/placementDS` : the pooled datasource, named in `persistence.xml` ; `PortalStatsBean` gets it by `@Resource` and by `InitialContext.lookup` .
- `java:module/ShortlistBean` : `new InitialContext().lookup( ... )` , `ShortlistServlet` line 135. The queue and its factory are `java:app/jms/ ...` names, created by annotations on `NotificationSender` .
- `@EJB` fills a field once when the object is created, right for stateless beans. A **lookup** asks when you need it and gives a new stateful instance each time, so `ShortlistBean` is looked up and kept in the HttpSession.

JPA mapping

- **Foreign keys.** The owning side has `@JoinColumn` and its table holds the key; the other side says `mappedBy` . `@OneToOne` `StudentProfile.user` owns `user_id` (unique). `@ManyToOne` `Drive.company` owns `company_id` , inverse `Company.drives` (`@OneToMany`). `@ManyToOne` `JobApplication.student` and `.drive` own `student_id` and `drive_id` . All four exist in MySQL.
- **No double application.** `JobApplication.java` line 26: `@UniqueConstraint(name = "uk_student_drive", columnNames = {"student_id", "drive_id"})` . `hasApplied` is the polite check; the database rule catches a replayed POST or a race.
- **EAGER and LAZY.** The three `@ManyToOne` links and the `@OneToOne` are EAGER, so a page can print `application.student.user.fullName` after the transaction ended. The three `@OneToMany` lists are LAZY and throw `LazyInitializationException` on a detached entity, so the beans count with queries instead.
- **Named queries.** 16 are defined (for example `JobApplication.exists`). Three are never called: `StudentProfile.byRoll` , `Company.byName` , `Drive.byCompany` .

persistence.xml

```
<persistence-unit name="placementPU" transaction-type="JTA">           line 18
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider> line 19
  <jta-data-source>java:app/jdbc/placementDS</jta-data-source>       line 26
  hibernate.hbm2ddl.auto = update                                    line 47
```

- Hibernate 6.4.4 (the server log says "Hibernate ORM core version 6.4.4.Final") does the mapping, not whatever the server bundles. JTA: GlassFish owns the transaction, Hibernate joins it. `update` adds missing tables on deploy and never drops data; `create-drop` would wipe it.

Entity lifecycle with a real example

- **Transient:** `new JobApplication( ... )`, `ApplicationService.java` line 79.
- **Managed:** after `em.persist` (line 80) or `em.find` in `updateStatus` (line 88). `setStatus` on a managed entity is enough: the UPDATE runs at commit.
- **Detached:** anything a bean returns, because the transaction is over. `driveService.find()` runs with no transaction, so the servlet gets a detached drive. `DriveService.save` uses `merge` for one with an id.
- **Removed:** `em.remove(application)` in `withdraw`, line 119. The DELETE runs at commit.

The eligibility engine, step by step

`EligibilityRules.check(student, drive, alreadyApplied, today)`, line 32. A null student or drive is rejected at once (line 36). Otherwise it never stops at the first failure: every failing rule adds a reason.

`DriveService.check()` (line 148) asks the database one question, `hasApplied`, then calls it with `LocalDate.now()`.

- **1 Drive status** (41): not OPEN. "This drive has been closed by the placement cell."
- **2 Deadline** (44): last date before today, the last day itself passes. "The last date to apply was 2026-08-29."
- **3 CGPA** (47): below the minimum, equal passes. "Requires a CGPA of 7.50, your record shows 5.60."
- **4 Backlogs** (52): above the limit, equal passes. "Allows at most 0 backlog(s), your record shows 3."
- **5 Branch** (57): the drive lists branches and none matches, ignoring case. Empty list means any. "Open to Mechanical, Civil, your branch is Computer Science."
- **6 Already applied** (64): a flag, not a rejection. `isEligible()` stays true, `isCanApply()` is false.

CAREFUL

There is no "already placed" rule. A student holding an offer can still apply. Placement status only feeds counts and displays (`countPlaced`, `placedStudentIds`). If asked, say so.

It also runs inside the POST in `ApplicationServlet` (line 90). `EligibilityRulesTest` has 10 tests; the whole suite is 146.

The audit trail

- `@AroundInvoke record()` wraps each method of a bean with `@Interceptors`: `ctx.proceed()` (line 52), times it, hands the row to `AuditWriter`.

- **Reads are skipped.** A name starting `has, for, list, find, count, placed, statusBreakdown, search, check` or ending `Taken` writes no row on success (lines 31, 69). A failing read is still recorded.
- **Entities print as Type#id** (`label()` , line 117), for example `StudentProfile#1, Drive#8` for `apply` . Strings, numbers and enums print as themselves.
- **Secrets are redacted:** a parameter named password, secret, token or credential prints `[redacted]` (line 131).
- The actor is always `portal` (line 149): sign in is by the app's own filter, so the audit does not record which officer acted.

The message driven bean path

- **Message and sender:** a text string such as "Aarti Deshpande moved from Applied to Shortlisted for Lumen Health", sent by `NotificationSender.send()` from `apply` , `updateStatus` , `withdraw` and `ShortlistBean.commit` .
- **Consumer:** `NotificationMDB.onMessage()` (line 43) logs "Notification delivered: ..." and saves `AuditLog("notifier", "NOTIFICATION", body)` , after the officer's request has returned.
- **Proof:** Audit page, search `NOTIFICATION` (actor `notifier`), or search `server\glassfish7\glassfish\domains\domain1\logs\server.log` for `Notification delivered` .

5. Code walk in VS Code

- 1 `persistence.xml` 18 to 47; `JobApplication.java` 26, 49 to 54; `StudentProfile.java` 39, 65.

Say: "Hibernate is the provider. This side owns the foreign keys, the unique key stops a double application, lists are lazy."

- 2 `ApplicationService.java` lines 24 to 32, `apply` 75, `updateStatus` 87. Say: "No begin or commit. The container makes the transaction, Hibernate saves lines 93 and 94 at commit, and a message is queued." Then `DriveService.java` 148, `EligibilityRules.java` 32 to 66, `EligibilityRulesTest.java` 139: "A static method, so tests need no server."

- 3 `NotificationSender.java` 26 to 51; `NotificationMDB.java` 27 to 52; `AuditInterceptor.java` 31, 45 to 57, 69, 117; `AuditWriter.java` 26. Say: "The sender creates its own queue, nobody calls the MDB, the container does. `@AroundInvoke` wraps the call, reads are skipped, entities print as Type#id, and `REQUIRES_NEW` lets the audit row survive a rollback."

- 4 `ShortlistBean.java` 35, 50, 52 to 60, 108, 131; `ShortlistServlet.java` 135 to 136, 99, 152; `PortalStatsBean.java` 33, 34, 45, 84. Say: "Stateful, one per officer, looked up by JNDI and kept in the HttpSession. Commit is one transaction, `@Remove` ends it. The singleton starts at deploy."

6. Live demo plan

Base `http://localhost:8080/placement` . Every password is `Campus@2026` . Officer in one browser, student in a private window. Seed: 25 students, 14 drives, 85 applications, 6 offers (Aarti holds none).

1 Not eligible, with reasons. Sign in as `aarti.deshpande@campus.edu` (Computer Science, CGPA 8.7, 0 backlogs). Open `/app/drive?id=9` (Ironbridge): branch No, "Open to Mechanical, Civil, your branch is Computer Science." Then `/app/drive?id=13` (Orbit, closed): three reasons at once. As `rohan.kapoor@campus.edu` (CGPA 5.6, 3 backlogs) open `/app/drive?id=4`: CGPA and backlog reasons.

2 Officer moves a status, audit shows it. Sign in as `tpo@campus.edu`, open **Pipeline** (`/admin/applications`), find Aarti Deshpande, Lumen Health, Applied (application 8), choose Shortlisted, add a remark, **Save**. Open **Audit** (`/admin/audit`): two rows, `ApplicationService.updateStatus` with detail `8, SHORTLISTED, your remark`, then actor `notifier`, `NOTIFICATION`, "Aarti Deshpande moved from Applied to Shortlisted for Lumen Health". Note the `[redacted]` rows and that browsing added none. Search `server.log` for `Notification delivered`. Then set application 8 back to Applied.

3 Withdraw refused, the exception story. As Aarti open **My applications** (`/app/applications`). Withdraw only shows on Applied rows. Press F12, change the hidden `applicationId` in an Applied row's Withdraw form to `1` (Shortlisted), then click Withdraw and accept the dialog. The flash says the placement cell has already progressed it, so it cannot be withdrawn; the row is unchanged. Audit shows `ApplicationService.withdraw`, `1, 1 [failed: IllegalStateException]`. Say: "The bean threw, the container rolled back and wrapped it in `EJBException`, and the servlet reads `getCause()`." **Click without editing the id and that application is deleted.**

4 The stateful basket. Officer, **Shortlist** (`/admin/shortlist`): pick a drive, tick two names, reload (ticks stay, nothing written), click **End the session**. `server.log` shows "ShortlistBean created" and "finish() called".

5 MySQL, joined tables, unique key. In PowerShell:

```
$env:MYSQL_PWD = 'Placement@2026'
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user placement_cell

SELECT a.id, u.full_name, c.name AS company, d.job_role, a.status
FROM job_application a
JOIN student_profile s ON s.id = a.student_id
JOIN app_user u      ON u.id = s.user_id
JOIN drive d         ON d.id = a.drive_id
JOIN company c       ON c.id = d.company_id
WHERE a.id = 8;
```

Say: "Four tables, four foreign keys, built by Hibernate from the annotations." `SHOW CREATE TABLE job_application\G` shows `UNIQUE KEY uk_student_drive (student_id, drive_id)`. MySQL times are UTC, 5 h 30 min behind the pages. JNDI is on `/admin/system`.

7. Questions you will be asked

Q Stateless versus stateful session bean?

Stateless keeps nothing between calls, so any pooled instance serves any caller (the three services). Stateful is one instance per client that remembers its fields (`ShortlistBean`).

Q Why is the shortlist bean stateful, and where is it held?

The officer ticks names across page loads, so the selection belongs to one person. The instance lives in the EJB container (30 minute timeout), the reference in that officer's HttpSession. A servlet field would be shared by all officers.

Q Singleton bean and @Startup? Interceptor?

A singleton is one instance for the whole app; `@Startup` creates it at deploy. An interceptor's `@AroundInvoke` method wraps a business method and calls `ctx.proceed()` , so the beans hold no audit code.

Q What is JNDI, and how does @EJB differ from a lookup?

JNDI is the container's name service: look up a name, get a resource, no URL or password in code. `@EJB` looks up once at creation; an explicit lookup gives a new stateful instance each time.

Q What does @PersistenceContext do?

It injects a container managed, transaction scoped `EntityManager` for `placementPU` . You never open or commit it.

Q Why Hibernate? JPA versus Hibernate?

JPA is the specification, Hibernate the implementation, named in `persistence.xml` . It writes the SQL, so only the dialect changes for another database.

Q Which side owns the foreign key, and how is a double application stopped?

The side with `@JoinColumn` , for example `JobApplication.student` . `hasApplied` checks first, and the unique key `uk_student_drive` refuses a second row even if the check is bypassed.

Q LAZY versus EAGER, and what is a detached entity?

EAGER loads the related row with the entity, LAZY waits until touched. A detached entity has lost its transaction, as in a servlet; a lazy list on it throws `LazyInitializationException` . `merge` reattaches it.

Q What happens when an exception is thrown inside a transaction?

A runtime exception is a system exception: the container rolls the transaction back and the caller gets it wrapped in `EJBException`, so the servlet reads `getCause()`, as `withdraw` does. The audit row survives (`REQUIRES_NEW`). Reads use `SUPPORTS`, writes `REQUIRED`.

Q How does the MDB work, and can a placed student apply again?

`NotificationSender.send` queues a text message; the container calls `NotificationMDB.onMessage`, which logs it and saves a NOTIFICATION row. A placed student can apply again: no rule forbids it.

8. If something goes wrong

SYMPTOM	CAUSE AND WHAT TO DO
No NOTIFICATION row	The MDB runs a moment later. Refresh, or check <code>server.log</code> .
Withdrew a real row	You clicked before editing the id. Tell Karan.
Server not answering	Tell Karan. Restarting never wipes data.



EVERYONE

Viva questions and answers

Every question we expect, who answers it, and a short correct answer. The owner's own brief has more detail.

How to answer

Answer in two or three sentences, then offer to show it: "I can open that file." Say what the code does, not what you wish it did. If a question is outside your layer, say "that is Chandra's part" and hand it over. If you do not know, say so and say where in the code you would look.

A. Questions that cross layers

Q What happens, step by step, when a student presses Apply?

KARAN STARTS, THEN TIYA, CHANDRA, YUVRAJ

1. The browser sends a POST to `/app/apply` with a hidden CSRF token. 2. `AuthFilter` checks there is a signed in session (and the officer role for `/admin/*`), and on a POST that the CSRF token matches. 3. `ApplicationServlet.doPost` loads the student and the drive and asks `DriveService.check` for the verdict. 4. If already applied or not eligible, it flashes a message and redirects. 5. Otherwise it calls `ApplicationService.apply`, a stateless EJB, inside a container managed transaction: it persists a `JobApplication` and queues a notification message. 6. Hibernate writes the row to MySQL, and the unique constraint on student and drive is the last guard. 7. The servlet redirects to `/app/applications`, which forwards to a JSP that lists the rows. 8. Meanwhile the message driven bean logs "Notification delivered".

Q Why do you redirect after a POST instead of showing the page directly? KARAN, TIYA

Post, Redirect, Get. If the browser were left on the POST result, pressing refresh would submit the form again and apply twice. A redirect makes the browser load a fresh GET, so refresh is harmless. The flash message survives the redirect because it is stored in the session and removed after it is shown once.

Q Why do you check eligibility both on the page and in the servlet? KARAN, CHANDRA

The page only hides the button. A page can be stale, a form can be replayed, and a request can be written by hand, so the rule check that matters is the one on the write path. The database adds a third guard, a unique constraint on student and drive.

Q You use both Hibernate and JDBC. Why? YUVRAJ, CHANDRA

Ordinary reads and writes are object shaped, so JPA with Hibernate is the right tool: one object, one row. Reports are the opposite: they aggregate across tables, and plain SQL says that more directly than mapped objects. Both use the same server managed connection pool by the same JNDI name, so there is one pool and one database.

Q Where is the database password kept? YUVRAJ

In `WEB-INF/glassfish-resources.xml`, which defines the connection pool and the datasource `java:app/jdbc/placementDS`. The application code asks for that name and never sees a URL or a password. The only exception is `MockScoreBean.addScore`, the classroom style bean, which has them written in the code on purpose, to show the difference.

Q How do the JSP pages get their data? KINJAL, KARAN

The servlet puts the data on the request as attributes, then forwards with `RequestDispatcher` to a JSP under `WEB-INF/views`. The JSP reads the attributes with EL and loops with JSTL. There is no Java in the page, because scriptlets are switched off in `web.xml`. A browser cannot ask for a JSP directly because files under `WEB-INF` are never served.

Q How is this different from the practical where we used a Stateful bean with DriverManager? YUVRAJ

The practical opened a connection with `DriverManager` with the URL and password in the code, joined the values into the SQL text, and never closed the connection. We keep that as `MockScoreBean.addScore` and run it live on the Classroom JDBC page. Beside it, `addScoreSafely` takes a connection from the pool, binds values with placeholders and closes everything. Both write to MySQL.

Q Which parts of the course syllabus did you use? KARAN

Servlets with lifecycle and annotations, `RequestDispatcher`, cookies, `HttpSession` and a listener, file upload and download, non blocking I/O, JSP with actions, EL and JSTL, EJB of every kind (stateless, stateful, singleton, message driven, interceptor), JNDI, JPA with Hibernate, and JDBC. The table in `00-start-here` shows where each one is.

Q What would you do next if you had more time? KARAN

A recruiter role with its own login and an approval step by the officer, real email delivery for notifications, password reset and email confirmation links, and paging on the long lists.

Q How did you test it? CHANDRA

146 automated JUnit tests covering the eligibility rules, validation, password hashing, email domain checks, uploads, and live database tests for the JDBC layer. They run with `.\test.ps1` and finish in a few seconds. The pages were also walked through in a browser.

B. Yuvraj: MySQL and JDBC

13 questions. The full brief is `02-yuvraj-mysql-jdbc`.

Q What happens if the password in the pool is wrong?

MySQL refuses the login with "Access denied for user placement_user". The password lives in one place, `glassfish-resources.xml` (line 52). The pool is set with `ping="true"`, so GlassFish tests it and reports the failure. In the app, `describeConnection()` catches the `SQLException` (lines 115 to 119) and the Reports card shows "Unavailable" with MySQL's message instead of crashing. Pages that go through Hibernate will also fail. Fix the password in the XML and redeploy with `.\run.ps1`. No Java file changes.

Q What is a connection pool?

A set of already open, already authenticated MySQL connections kept by the server. Code borrows one with `getConnection()` and gives it back with `close()`. Ours (lines 41 to 47): 4 connections opened to start, at most 24, idle ones dropped after 300 seconds, and each is validated before it is handed out.

Q Why not open a connection per request?

Opening a TCP connection and logging in to MySQL costs far more than the query itself, and MySQL allows only a limited number of connections. A pool pays that cost once and reuses. That is exactly what `MockScoreBean.addScore` does wrong: it opens a new connection on every call and never closes it.

Q Is this safe from SQL injection? Show me.

Yes for values. Show `searchCandidates` lines 271 to 299: the SQL only ever gets `?` added, and values are bound with `setObject`. Then run the mysql `PREPARE` demo: concatenated returns 26 rows, bound returns 0. Two honest limits. A table name cannot be a `?`, so `requireKnownTable` (explorer, line 347) accepts only names the database itself reported, and `countRows` (report DAO, 426 to 437) uses a fixed list. And the classroom method is unsafe on purpose, which is why the servlet validates first.

Q What goes wrong with the classroom version?

Four things, all visible in `addScore`. It opens a new connection on every call and never closes it, so connections leak. The URL and password are in the source. The SQL is built by joining strings, so a name like `O'Brien` ends the string early and the statement is rejected, which is SQL injection in its everyday form. And errors are only printed: the practical hides the failure completely, ours returns 0 so the page shows an error, but the exception itself still only goes to the server log. The pooled method saves the same name because the value is bound, not joined.

Q What is the difference between Statement and PreparedStatement?

A Statement sends the finished SQL text. A PreparedStatement sends the SQL with `?` placeholders, and the values travel separately, typed, so they can never be read as SQL. It also lets the database reuse the parsed statement. We still use a plain Statement in a few places where no user value is joined in: row counts over a fixed or database-reported table list, and the console, which runs officer typed reads behind the guard.

Q What if you forget to close the connection?

With a pool, `close()` does not close the socket, it returns the connection. Forget it and the connection is never returned. After 24 leaks the pool is empty and every request waits. That is why every connection here is in a try with resources block, and why `closeDriverAndRejectPending` closes in a `finally`.

Q Why JDBC for reports if you already have Hibernate?

Hibernate is right for loading an object, changing a field and saving it. Reports aggregate across tables and return numbers, never written back. Plain SQL with `COUNT`, `GROUP BY` and a LEFT JOIN is clearer than forcing it into JPQL. Both use the same datasource, so it is one pool, one set of credentials.

Q Explain the transaction in closeDriverAndRejectPending.

Two updates must succeed together: set the drive to CLOSED, and set its APPLIED applications to REJECTED. `setAutoCommit(false)`, run both, `commit()`. On any `SQLException`, `rollback()` undoes both. The finally block restores auto commit, then closes. The bean is `NOT_SUPPORTED` so the container does not start its own JTA transaction, which would conflict with one driven by hand.

Q Can someone destroy data through the SQL console?

Three layers. One: the guard (must start with select, with, show, describe or explain, one statement, no forbidden keyword such as drop, update or into). Two: `setReadOnly(true)` on the connection, so the server refuses writes even if the guard were fooled. Three: the database user only has rights on `placement_cell`. Be honest about layer three: that user does have all privileges on this schema, so layers one and two are what stop writes here. Rows are capped at 200 and a query is cancelled after 5 seconds.

Q Why is the MySQL driver not inside the WAR? Do you still need Class.forName?

The pool is created by GlassFish before the application loads, so the driver must be on the server's classpath: `run.ps1` copies it into `domain1\lib`. And `Class.forName` is not needed with a modern JDBC 4 driver, which registers itself. It appears in `MockScoreBean.addScore` only because the practical uses it.

Q Where is DatabaseMetaData used?

Two places. `describeConnection()` (report DAO, 100 to 121) fills the "live connection" card: product name and version, driver, URL, user, isolation. `describe()` (explorer DAO, 99 to 170) reads columns, primary keys, foreign keys and indexes for the Structure tab. `tableNames()` uses `getTables` as the whitelist.

Q The password is in plain text in an XML file. Is that acceptable?

It is a local database made for this project and the user only has rights on one schema. It is listed as an accepted risk in `docs/SECURITY.md` (item A1). The production step would be a GlassFish password alias or a secret from the environment. The point of the design is that it is in one server file, not compiled into classes.

C. Tiya: Accounts and security

13 questions. The full brief is `03-tiya-auth-security`.

Q How is an account created?

Student self sign up only, at `/register`. `RegisterServlet` checks the CSRF token, name, email (four checks), password strength, roll number, branch, batch year, CGPA, backlogs and phone. Only if there are no errors does it call `AuthService.registerStudent`, which checks for a duplicate email or roll number again, hashes the password, and saves an `app_user` row plus a student profile. The role is hard coded to STUDENT. Officers exist only through the seeder, never through sign up.

Q Would it accept a fake email?

It rejects an address whose domain does not exist: `test@thisdomaindoesnotexist123.com` gets "That domain doesn't appear to accept email". It rejects a malformed one such as `admin@campus`, and 43 known disposable providers such as mailinator. It would still accept an invented mailbox at a real domain, for example `asdf1234@gmail.com`, because we do not check the mailbox.

Q So do you verify the email really exists?

Done: shape check (`Validators.isEmail`), disposable blocklist, and a DNS lookup for an MX record with A and AAAA as fallback (`EmailDomainVerifier`). That proves the domain can receive mail. **Not done:** no confirmation link, no OTP, no test send, and nothing proves the person owns the mailbox. That needs an SMTP account we do not have. It is the recorded next step (Security review S10). If DNS cannot be reached at all, the check lets the sign up through and logs a warning, on purpose.

Q Does it hide the password?

Yes on screen: the inputs are `type="password"` on both forms. The forms use POST, so the password is in the body, not the URL. When a form fails, the servlet forwards the page back with no password value, so the boxes come back empty. The audit trail records the call as `authenticate(email, [redacted])`. Honest limit: the app runs on plain HTTP on localhost, so on a real network it would need HTTPS.

Q How is the password stored? Can you see it?

As `pbkdf2$120000$salt$hash` in `app_user.password_hash`: PBKDF2 with HMAC SHA 256, 120,000 rounds, a 16 byte random salt per account, 256 bit output (`PasswordHasher`). It is one way, so we cannot read the original. Login hashes the typed password with the stored salt and compares in constant time. The salt means two people with the same password get different hashes.

Q What happens after wrong passwords?

Each wrong password adds one to `failed_attempts` in the database (`AuthService` lines 74 to 85). The message is "Email or password is incorrect." with HTTP 401. With two or one attempts left it adds a countdown. The fifth sets `locked_until` fifteen minutes ahead. While locked, even the correct password is refused. A correct sign in resets the counter. The officer can clear a lockout early on the student's page. The lock lives in the database, so restarting the server does not clear it, and it counts per account, not per computer.

Q Can someone tell which emails have accounts?

Not from the wrong password message: unknown email and wrong password read the same, and an unknown email still burns one hash so the timing matches. Two small leaks exist and you should say so: the countdown and lockout messages only appear for real accounts, and the register page says "An account already exists for that address".

Q Can a student open the admin pages?

No. `AuthFilter` line 63 checks the role for every path under `/admin`, and a student gets HTTP 403 with the "Not allowed" page. Someone not signed in is redirected to login. The path it tests is cleaned first by `RequestPaths.canonical`, so `//admin/database` and `/admin;x=1/students` are refused too. We tested all of these as a student. This was a real bug in an earlier version: the check read the raw URL, the container routed on the cleaned one, and a doubled slash got through. It is fixed and covered by `RequestPathsTest`.

Q What is session fixation and how do you stop it?

An attacker plants a known session id in the victim's browser, waits for the victim to sign in, then uses the same id. We call `request.changeSessionId()` right after a successful password check (`LoginServlet` line 98), so the pre login id becomes useless. Only cookies are allowed for the id (`web.xml` line 30), so it cannot sit in a URL.

Q How long does a session last, and who counts them?

30 minutes of inactivity: `web.xml` line 23, and `SessionTracker` line 24 sets the same 30 minutes when each session is created. Logout invalidates it at once. `SessionTracker` implements `HttpSessionListener`; the container calls it on creation and on invalidation or timeout, and the officer's System page shows "Sessions open now".

Q Which cookies do you set, and with what flags?

`JSESSIONID`: `HttpOnly` (in `web.xml`). `remember_email` (only if the box is ticked) and `last_visit`: `HttpOnly`, 30 days, path `/placement`, values percent encoded. `portal_theme`: one year, not `HttpOnly`, only says light or dark. No cookie holds a password. Not set: the `Secure` flag, because a secure cookie would never be sent over plain HTTP on localhost (accepted risk A2 in the review), and no `SameSite` attribute is set in our code.

Q Why are the JSPs under WEB-INF? Forward or redirect?

Anything under `WEB-INF` cannot be requested by URL: we tested `/placement/WEB-INF/views/login.jsp` and got 404. Only a servlet can reach a view, by `RequestDispatcher.forward`, which happens inside the server: the address bar keeps the servlet URL. A redirect sends the browser a 302 and it makes a second request. We forward to show a page, including a failed login, and redirect after every successful POST (Post-Redirect-Get) so refresh cannot repeat the action.

Q What are CSRF and the Content Security Policy?

CSRF: another site making your browser submit a form to us. Every POST needs the random per session token in a hidden field (`Web.csrfValid`). **CSP:** `script-src 'self'` with no inline scripts, so the browser refuses any script we did not serve. Plus `frame-ancestors 'none'` and `X-Frame-Options: DENY` against clickjacking, `form-action 'self'`, `nosniff`, and no-store on every page that is not a static asset. Not set: HSTS, because we have no HTTPS.

D. Kinjal: Views, validation and uploads

14 questions. The full brief is `04-kinjal-views-validation`.

Q Why can't I open your JSPs directly?

They live under `WEB-INF/views`, and the container never serves anything under WEB-INF to a browser. The only way in is `RequestDispatcher.forward` from a servlet, which is what `Web.render` does. Try it: `/placement/WEB-INF/views/drives.jsp` gives a 404. The one exception is `index.jsp`, the welcome file, and it holds no markup.

Q Why are scriptlets disabled, and where?

In `web.xml`, `<scripting-invalid>true</scripting-invalid>` for `*.jsp` and `*.jspx`. Java inside a page mixes logic into the view, cannot be tested, and is where injection bugs hide. With it off, a page containing a scriptlet fails to compile, so the rule cannot be broken by accident.

Q Include directive or `jsp:include` ? Which do you use?

The include directive, `<%@ include file="layout-top.jspf" %>`. It is a static include: the fragment is pasted in when the page is translated, so the whole page becomes one servlet and variables like `ctx` and `me` set in the fragment are visible to the page. `jsp:include` would run at request time as a separate resource. We do not use it.

Q What does `c:out` do, and why is it on every user value?

It escapes `& < > " '` before printing. Without it, a name like `"><script>alert(1)</script>` would be written into the page as live markup. With it, the register page sends back `"><script>alert(1)</script>`, which the browser shows as plain text. Plain EL like `${x}` does not escape. I checked this against the running register page.

It is used inside attributes too, not only in page text. The `data-confirm` messages that contain a name (`admin-companies.jsp` line 52, `admin-student-detail.jsp` lines 101 and 129, `my-applications.jsp` line 52) use `<c:out value='${co.name}' />` inside the attribute. Plain EL remains only for numbers and server made values such as ids, the csrf token and status labels. One client supplied exception: the theme cookie value on the html tag, which only the person's own browser can set.

Q How does a servlet hand data to a JSP?

It calls `request.setAttribute("drives", drives)` and then forwards. The JSP reads `${drives}`. EL searches page, request, session, then application scope for the name. The address bar keeps the servlet URL because a forward happens inside the server.

Q Forward or redirect, and when?

Forward to show a page: same request, attributes survive, no second round trip. Redirect after a successful POST so a refresh does not repeat the write. A message that must survive the redirect goes in the session as `flashSuccess` or `flashError`, and `layout-top.jspf` removes it with `c:remove` after printing.

Q Is validation done in the browser or on the server?

Both, but only the server counts. Inputs carry `required`, `min`, `max`, `maxlength`, `type="email"`, and the forms also carry `novalidate`, so the browser does not block a submit or show its own bubbles. `Validators` does the real check and the page shows the message under the field with `aria-invalid="true"`. The register page says it plainly: "Every field is checked again on the server before anything is saved." Anyone can bypass a browser with curl.

Q Why is there no inline JavaScript?

The Content-Security-Policy header says `script-src 'self'` with no `'unsafe-inline'`, so the browser runs only scripts served from our own origin. An injected script tag would sit there unused. That only works if no page has a script block or an onclick, so everything is in `app.js`. Confirm dialogs use a `data-confirm` attribute and one listener. Inline *styles* are allowed by the policy and a few pages use them.

Q How does light and dark mode work?

Clicking the icon calls `/theme?mode=dark`. `ThemeServlet` stores `portal_theme` in a one year cookie, accepting only `light` or `dark`. Each page reads it with `cookie.portal_theme.value` and puts it on the html tag as `data-theme`. `app.css` line 102 swaps the colour tokens under `[data-theme="dark"]`. It does not follow the operating system setting.

Q How is the resume upload kept safe?

Four checks and one placement rule (plus a friendly message for an oversize file). `@MultipartConfig` caps a file at 2 MB and a request at 3 MB. The extension must be pdf, doc or docx. The first bytes must match that format (`%PDF` , or the zip or OLE signature). An oversize file is caught and answered with a normal message (`ProfileServlet` lines 73 to 82, `AuthFilter` lines 70 to 80). The saved name is generated (`resume-1-a1b2c3d4.pdf`), so the browser's file name is never used as a path. The file goes to `domain1/placement-resumes` , outside the deployed app, so a redeploy does not wipe it.

Q How is a resume served back, and can a student read someone else's?

Through `/app/resume` , not a static link. The servlet finds the file name from the student record, resolves it inside the upload folder only, and sends it with `Content-Disposition: attachment` and `nosniff` . A student asking for another student's id gets 403. The officer may fetch any.

Q Explain the non blocking bulk import in plain words.

Normally `read()` makes a server thread wait for the next byte, so a slow upload ties up a thread doing nothing. In `BulkImportServlet` the request is put in async mode (line 67) and a `ReadListener` is attached (line 73). The container calls `onDataAvailable` only when bytes are ready, we read while `isReady()` is true, then the thread goes back to the pool. When the body ends, `onAllDataRead` validates the rows, writes JSON and calls `async.complete()` . Limits: 512 KB and a 30 second timeout. The non blocking part is reading the request. The database work for each row still runs on that callback thread.

Q Is the import a file upload?

No. The officer pastes text into a box and `app.js` sends it with `fetch` as `text/csv` . It is not multipart, which is why it is a different mechanism from the resume upload. The CSRF token goes in the query string because there is no form body to carry it. The script writes the JSON reply into the page with `textContent` , never `innerHTML` .

Q What accessibility work did you do?

What is in the markup: `lang="en"` on every page; a `label for` on form fields; `aria-invalid` set by the server with visible error text, so colour is not the only signal; `aria-hidden` on the logo and icon sprite; `aria-label="Menu"` on the menu toggle; a visible `:focus-visible` outline, including on the CSS only menus; transitions only when the user has not asked for reduced motion; and a written contrast audit in the comments of `app.css` . Not done: no skip link, and the theme and sign out icons have `title` text but no `aria-label` .

E. Chandra: Entities and EJB

10 questions. The full brief is `05-chandra-entities-ejb` .

Q Stateless versus stateful session bean?

Stateless keeps nothing between calls, so any pooled instance serves any caller (the three services). Stateful is one instance per client that remembers its fields (`ShortlistBean`).

Q Why is the shortlist bean stateful, and where is it held?

The officer ticks names across page loads, so the selection belongs to one person. The instance lives in the EJB container (30 minute timeout), the reference in that officer's HttpSession. A servlet field would be shared by all officers.

Q Singleton bean and @Startup? Interceptor?

A singleton is one instance for the whole app; `@Startup` creates it at deploy. An interceptor's `@AroundInvoke` method wraps a business method and calls `ctx.proceed()` , so the beans hold no audit code.

Q What is JNDI, and how does @EJB differ from a lookup?

JNDI is the container's name service: look up a name, get a resource, no URL or password in code. `@EJB` looks up once at creation; an explicit lookup gives a new stateful instance each time.

Q What does @PersistenceContext do?

It injects a container managed, transaction scoped `EntityManager` for `placementPU` . You never open or commit it.

Q Why Hibernate? JPA versus Hibernate?

JPA is the specification, Hibernate the implementation, named in `persistence.xml` . It writes the SQL, so only the dialect changes for another database.

Q Which side owns the foreign key, and how is a double application stopped?

The side with `@JoinColumn` , for example `JobApplication.student` . `hasApplied` checks first, and the unique key `uk_student_drive` refuses a second row even if the check is bypassed.

Q LAZY versus EAGER, and what is a detached entity?

EAGER loads the related row with the entity, LAZY waits until touched. A detached entity has lost its transaction, as in a servlet; a lazy list on it throws `LazyInitializationException` . `merge` reattaches it.

Q What happens when an exception is thrown inside a transaction?

A runtime exception is a system exception: the container rolls the transaction back and the caller gets it wrapped in `EJBException`, so the servlet reads `getCause()`, as `withdraw` does. The audit row survives (`REQUIRES_NEW`). Reads use `SUPPORTS`, writes `REQUIRED`.

Q How does the MDB work, and can a placed student apply again?

`NotificationSender.send` queues a text message; the container calls `NotificationMDB.onMessage`, which logs it and saves a NOTIFICATION row. A placed student can apply again: no rule forbids it.



EVERYONE

Demo run sheet

One new student is followed through every layer of the app, so each part of the presentation hands over to the next.

22

minutes of demo

6

speakers, in order

1

student followed end to end

The demo student

Tiya creates this account live. Nobody else uses it before then, and **-Reseed** removes it afterwards.

FIELD	TYPE EXACTLY
Full name	Demo Student
Email	demo.student.cs@gmail.com (a real domain, so the domain check passes)
Password	Demo@2026x
Roll number	CS26100
Branch, year	Computer Science, 2026
CGPA, backlogs	8.0, 0
Mobile	9876543210

With CGPA 8.0 and no backlogs, this student qualifies for several drives and is refused by others for a clear reason, which is exactly what Chandra needs to show.

The order

TIME	WHO	WHAT	SCREEN
0:00	Karan	What the app is, who uses it, how data moves in and out, the architecture picture	Login page, then 00-start-here diagram
2:30	Tiya	Creating an account: fake email refused, valid signup, password hash in MySQL, wrong password, sign in, student blocked from admin	Browser, then MySQL
7:00	Kinjal	The pages and the validation: an error on the profile form, resume upload, how a JSP gets its data	Browser, then VS Code

10:30	Chandra	Drives board and the eligibility engine, apply, officer moves the status, audit row appears, EJB types	Browser (two windows), then VS Code
15:00	Yuvraj	The connectivity story: the row Chandra just created, found in MySQL; Reports; Database page; Classroom JDBC against the practical sample	VS Code, browser, MySQL
20:00	Karan	Close: the officer overview now shows the new application, what was not built, questions	Officer overview

The hand overs

- 1 Karan to Tiya.** "To use any of this, someone has to get an account. Tiya will create one live."
- 2 Tiya to Kinjal.** "The student is signed in and lands on a dashboard with a getting started checklist that says step 2, finish your profile, is next. Kinjal will show how the pages work and finish the profile."
- 3 Kinjal to Chandra.** "With a complete profile the student can look at the drives. Chandra will show how the app decides what they can apply for."
- 4 Chandra to Yuvraj.** "That application is now a row in MySQL. Yuvraj will show how the app reaches the database and prove the row is there."
- 5 Yuvraj to Karan.** "Karan will close by showing the officer's overview, which now counts this application."

Before you start: setup checklist

WHEN	WHO	DO THIS
Sunday night	Karan	Pull the latest code. Run the whole demo once, start to finish, on the presenting laptop. Time it.
Monday, before leaving	Karan	Start MySQL. Run <code>.\run.ps1 -Reseed</code> and wait for "deployed". Fresh dates, and the demo student does not exist yet.
Monday, at the venue	Karan	Open <code>http://localhost:8080/placement/</code> . Sign in as the officer once, then sign out, to prove it works.
Monday, at the venue	Yuvraj	Open MySQL Workbench or a terminal on <code>placement_cell</code> with the queries from <code>demo-queries.sql</code> ready.
Monday, at the venue	Everyone	VS Code open on <code>placement-cell</code> with your own files already open in tabs, in the order you will show them.

Monday, at the venue	Chandra	Two browser windows: one normal for the student, one private for the officer, so the two sessions do not collide.
Monday, at the venue	Chandra	A spare terminal running <code>Get-Content server\glassfish7\glassfish\domains\domain1\logs\server.log -Wait -Tail 3</code> . When the student applies, a line <code>Notification delivered: Demo Student applied to ...</code> appears: that is the message driven bean.
Monday, at the venue	Yuvraj	Before you begin, the table <code>mock_score</code> does not exist. The first classroom save creates it, which is worth saying aloud.

CAREFUL

Do not press the reseed on the presenting laptop once the demo has begun: it deletes the demo student and the application you are about to show. If something breaks in the middle, reload the page first. Only reseed between runs.

If the projector, network or server fails

PROBLEM	DO THIS
Server not answering	<code>.\run.ps1</code> in the project folder, wait about a minute. Meanwhile talk through the architecture picture.
Email domain check is slow or fails	It looks up the domain's mail records on the network. Without internet, a valid domain can be refused. Use an account from the seeded list instead and explain the check using the code.
Everything is broken	Screenshots of every step of this demo are in <code>docs/team/backup-screenshots/</code> , numbered in the order of the presentation. Show those and the code.

What each person shows, in one line

PERSON	THE ONE THING THAT MUST LAND
Karan	Two kinds of user, and how information flows in and out.
Tiya	A fake email is refused, the password is never stored as text, and a student cannot enter the officer pages.
Kinjal	Pages contain no Java, every value is escaped, and the server re-checks what the browser checked.
Chandra	The business rules live in EJB beans, and every write is audited without the beans knowing.
Yuvraj	The app reaches MySQL through a pool by name, and the row created a moment ago is in the table.



EVERYONE, KEEP THIS OPEN DURING THE DEMO

Pastables cheat sheet

Every value the demo needs, exactly as written in each brief, in the order the run sheet uses them. Copy from here instead of typing from memory.

One account runs through the whole presentation. Karan opens the register page, Tiya creates the account below, and Kinjal, Chandra and Yuvraj all continue working with this same student afterward. Nobody invents a different one mid demo.

0. The one demo student, start to finish

FIELD	PASTE THIS
Full name	Demo Student
Email	demo.student.cs@gmail.com
Password	Demo@2026x
Roll number	CS26100
Branch	Computer Science
Graduating year	2026
CGPA	8.0
Live backlogs	0
Mobile number	9876543210

1. Karan: opening

WHAT	PASTE THIS
App URL	http://localhost:8080/placement/
Officer login	tpo@campus.edu / Campus@2026
Student login (seeded, not the demo student)	aarti.deshpande@campus.edu / Campus@2026

2. Tiya: account creation and security

STEP	PASTE THIS
Fake domain (rejected)	<code>demo.student@thisdomaindoesnotexist123.com</code>
Disposable domain (optional, rejected)	<code>x@mailinator.com</code>
Real sign up	the account in section 0 above
Wrong password to try	<code>Wrong@123</code>
Another unknown email (same refusal message)	<code>nobody.here@gmail.com</code>
Admin page as the student (403)	<code>http://localhost:8080/placement/admin/database</code>
Same page, the bypass that is now fixed (still 403)	<code>http://localhost:8080/placement//admin/database</code>
MySQL: show the new hash	run query 1 in <code>demo-queries.sql</code>

PowerShell, if you show the hash without opening the SQL file:

```
$env:MYSQL_PWD = 'Placement@2026'
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user placement_cell -e "SELECT id, email, user_role, password_hash FROM app_user ORDER BY id DESC LIMIT 3;"
```

3. Kinjal: views, validation, uploads

STEP	PASTE THIS
Trigger a validation error (CGPA out of range)	type <code>11</code> into CGPA on the profile form
Officer login for the import demo	<code>tpo@campus.edu</code> / <code>Campus@2026</code>
Import page	<code>http://localhost:8080/placement/admin/import</code>
Fake import row (a different person, not the demo student)	<code>Priya Patil,priya.patil.import@campus.edu,CS22099,Computer Science,2027,7.5,0,9820100299</code>

Role guard as student `http://localhost:8080/placement/admin/import`

4. Chandra: entities, EJB, eligibility

STEP	PASTE THIS
Drives board as the demo student	<code>http://localhost:8080/placement/app/drives</code>
Officer moves the application on	<code>http://localhost:8080/placement/admin/applications</code>
Audit trail	<code>http://localhost:8080/placement/admin/audit</code>
MySQL: the new application row, joined	run query 3 in <code>demo-queries.sql</code>
MySQL: the audit trail	run query 4 in <code>demo-queries.sql</code>

CAREFUL

The withdraw-a-progressed-application demo edits a hidden form field in devtools. Only do this on the demo student's own row, never on a seeded student's real application.

5. Yuvraj: MySQL and JDBC

STEP	PASTE THIS
Reports page	<code>http://localhost:8080/placement/admin/reports</code>
Database page	<code>http://localhost:8080/placement/admin/database</code>
Classroom JDBC page	<code>http://localhost:8080/placement/admin/classroom-jdbc</code>
Name that breaks the classroom (unsafe) method	<code>0'Brien</code>
Scores to enter with it	<code>70</code> , <code>75</code> , <code>80</code>
MySQL: table shapes and the unique key	run queries 5 and 6 in <code>demo-queries.sql</code>
MySQL: the mock_score table Classroom JDBC writes to	run query 7 in <code>demo-queries.sql</code>

6. Karan: closing

STEP	PASTE THIS
Officer overview, now showing the new application	<code>http://localhost:8080/placement/admin/dashboard</code>
MySQL: the totals the overview shows	run query 8 in <code>demo-queries.sql</code>

If something needs resetting

WHAT	DO THIS
The database password when MySQL asks	<code>Placement@2026</code>
Clear a lockout on the demo student	sign in as <code>tpo@campus.edu</code> , open Students, open the account, "Clear the lockout"
Remove the demo student after rehearsing	sign in as <code>tpo@campus.edu</code> , open Students, open the account, "Remove from the register". Never a direct database delete.

Full detail behind every one of these lines is in each person's own brief and in `07-demo-run-sheet` . This page only exists so nobody has to type a value from memory while presenting.